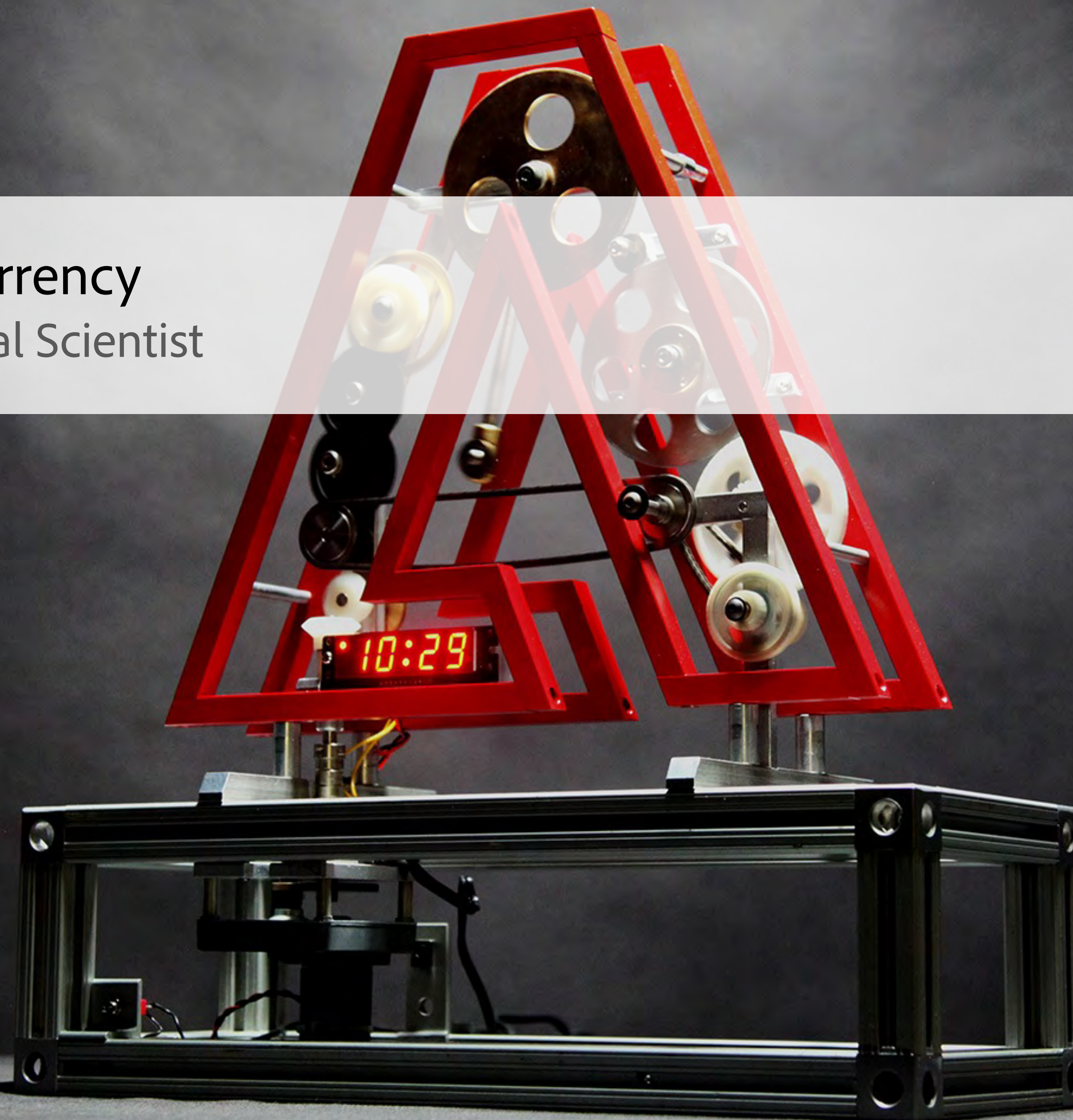


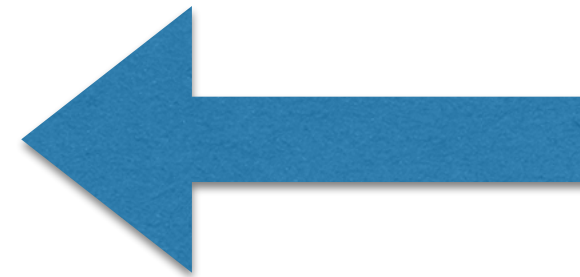


Better Code: Concurrency

Sean Parent | Principal Scientist



- Regular Type
 - Goal: Implement Complete and Efficient Types
- Algorithms
 - Goal: No Raw Loops
- Data Structures
 - Goal: No Incidental Data Structures
- Runtime Polymorphism
 - Goal: No Inheritance
- Concurrency
 - Goal: No Raw Synchronization Primitives
- ...



Common Themes

- Manage Relationships
- Understand the Fundamentals
- Code Simply
- Local and Equational Reasoning

lightroom.adobe.com

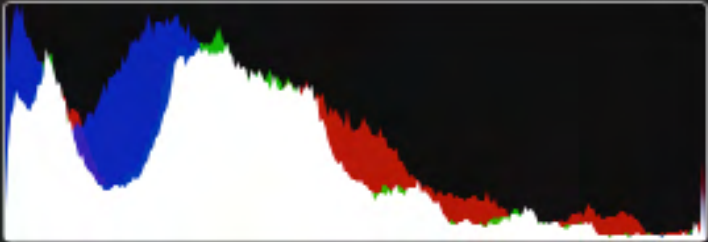
Revert to OriginalUndo Edits

Save & Exit

CROP PRESETS ADJUST



HISTOGRAM



TREATMENT

Color | Black & White

WHITE BALANCE

TONE

Auto

Exposure

Contrast

Highlights

Shadows

Whites

Blacks

PRESENCE

Clarity

Vibrance

Saturation

COLOR / B&W

SPLIT TONING

Demo

- Concurrency: when tasks start, run, and complete in overlapping time periods
- Parallelism: when two or more tasks execute simultaneously
- Why?
 - Enable performance through parallelism
 - Improve interactivity by handling user actions concurrent with processing and IO

Goal: No Raw Synchronization Primitives

What are raw synchronization primitives?

- Synchronization primitives are basic constructs such as:
 - Mutex
 - Atomic
 - Semaphore
 - Memory Fence
 - Condition Variable

You Will Likely Get It Wrong

Problems with Locks

```
template <typename T>
class bad_cow {
    struct object_t {
        explicit object_t(const T& x) : data_m(x) {}
        atomic<int> count_m{1};
        T data_m; };
    object_t* object_m;
public:
    explicit bad_cow(const T& x) : object_m(new object_t(x)) { }
    ~bad_cow() { if (0 == --object_m->count_m) delete object_m; }
    bad_cow(const bad_cow& x) : object_m(x.object_m) { ++object_m->count_m; }

    bad_cow& operator=(const T& x) {
        if (object_m->count_m == 1) object_m->data_m = x;
        else {
            object_t* tmp = new object_t(x);
            --object_m->count_m;
            object_m = tmp;
        }
        return *this;
    }
};
```

- There is a subtle race condition here:
- if count != 1 then the bad_cow could also be owned by another thread(s)
- if the other thread(s) releases the bad_cow between these two atomic operations
- then our count will fall to zero and we will leak the object

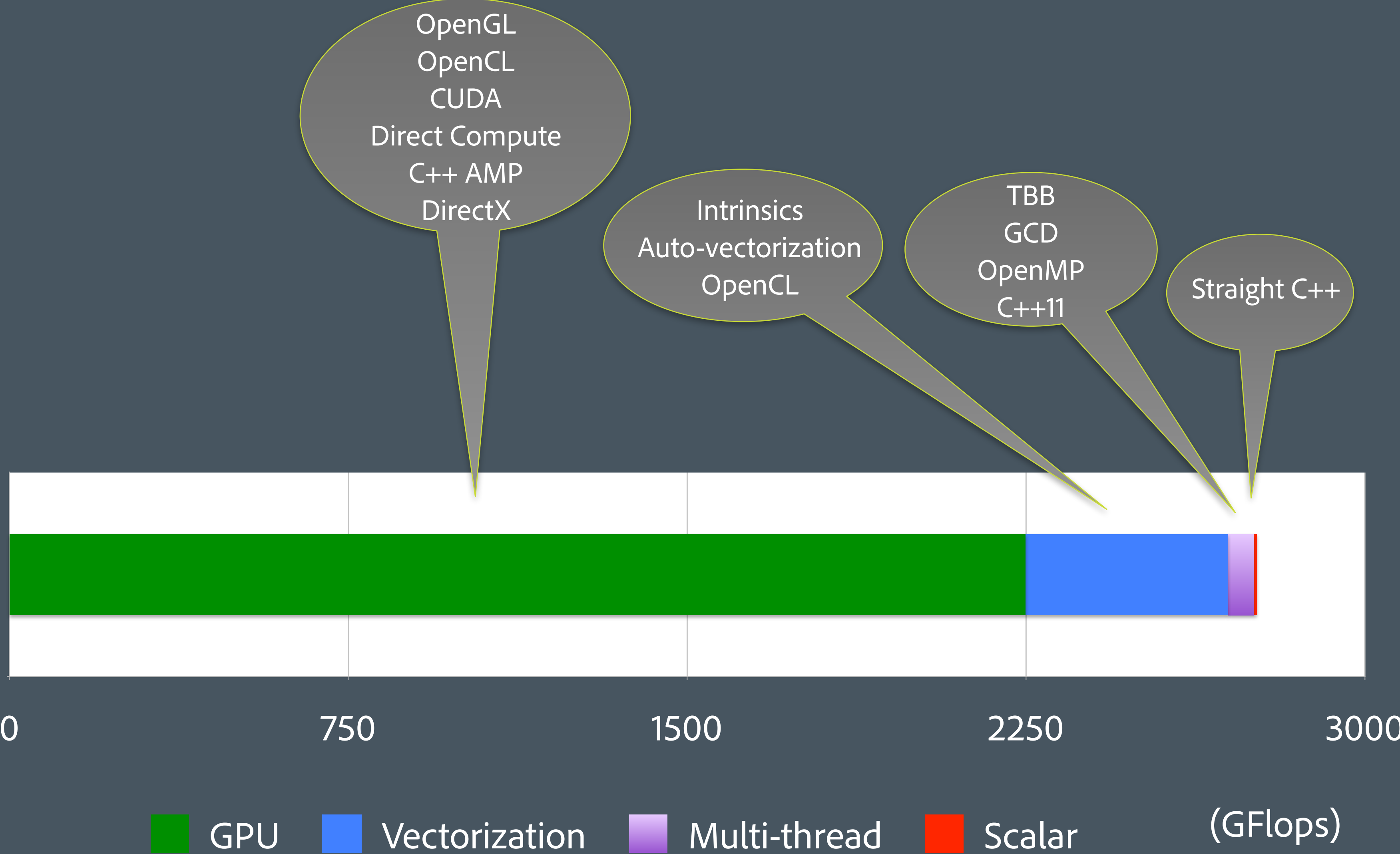
Problems with Locks

```
template <typename T>
class bad_cow {
    struct object_t {
        explicit object_t(const T& x) : data_m(x) {}
        atomic<int> count_m{1};
        T data_m; };
    object_t* object_m;
public:
    explicit bad_cow(const T& x) : object_m(new object_t(x)) { }
    ~bad_cow() { if (0 == --object_m->count_m) delete object_m; }
    bad_cow(const bad_cow& x) : object_m(x.object_m) { ++object_m->count_m; }

    bad_cow& operator=(const T& x) {
        if (object_m->count_m == 1) object_m->data_m = x;
        else {
            object_t* tmp = new object_t(x);
            if (0 == --object_m->count_m) delete object_m;
            object_m = tmp;
        }
        return *this;
    }
};
```

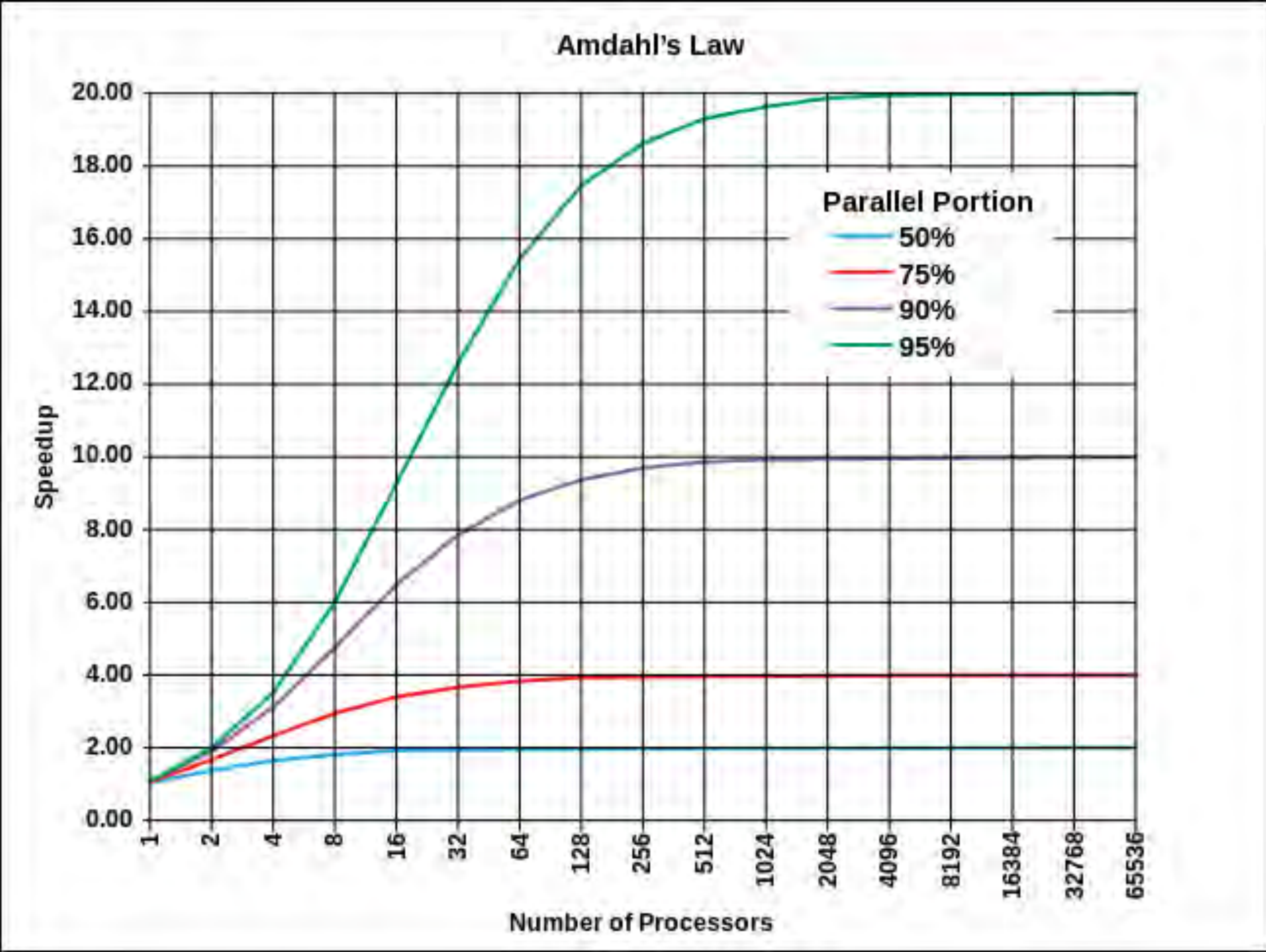

Performance through Parallelism

Desktop Compute Power (8-core 3.5GHz Sandy Bridge + AMD Radeon 6950)



Amdahl's Law

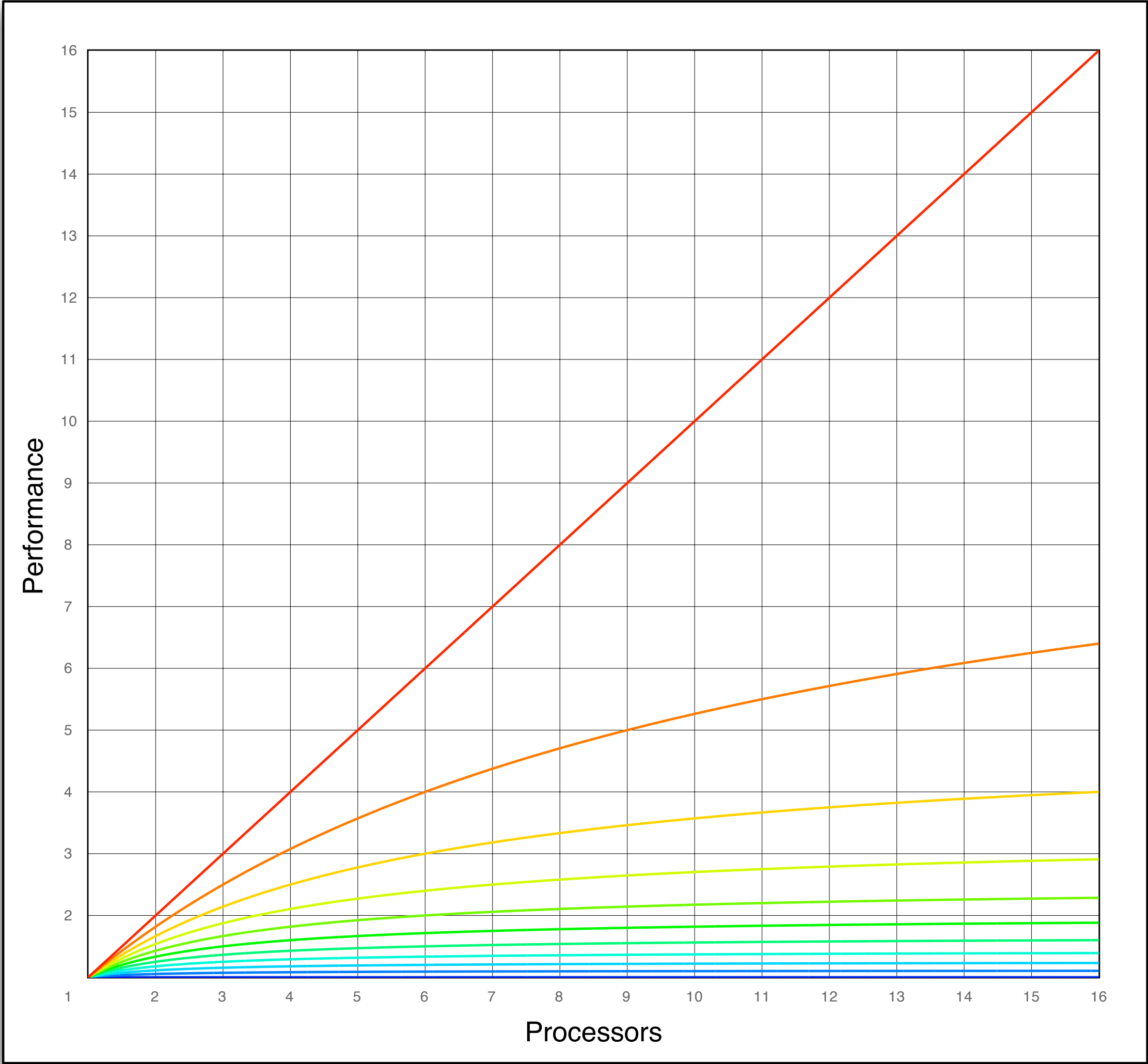
$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$



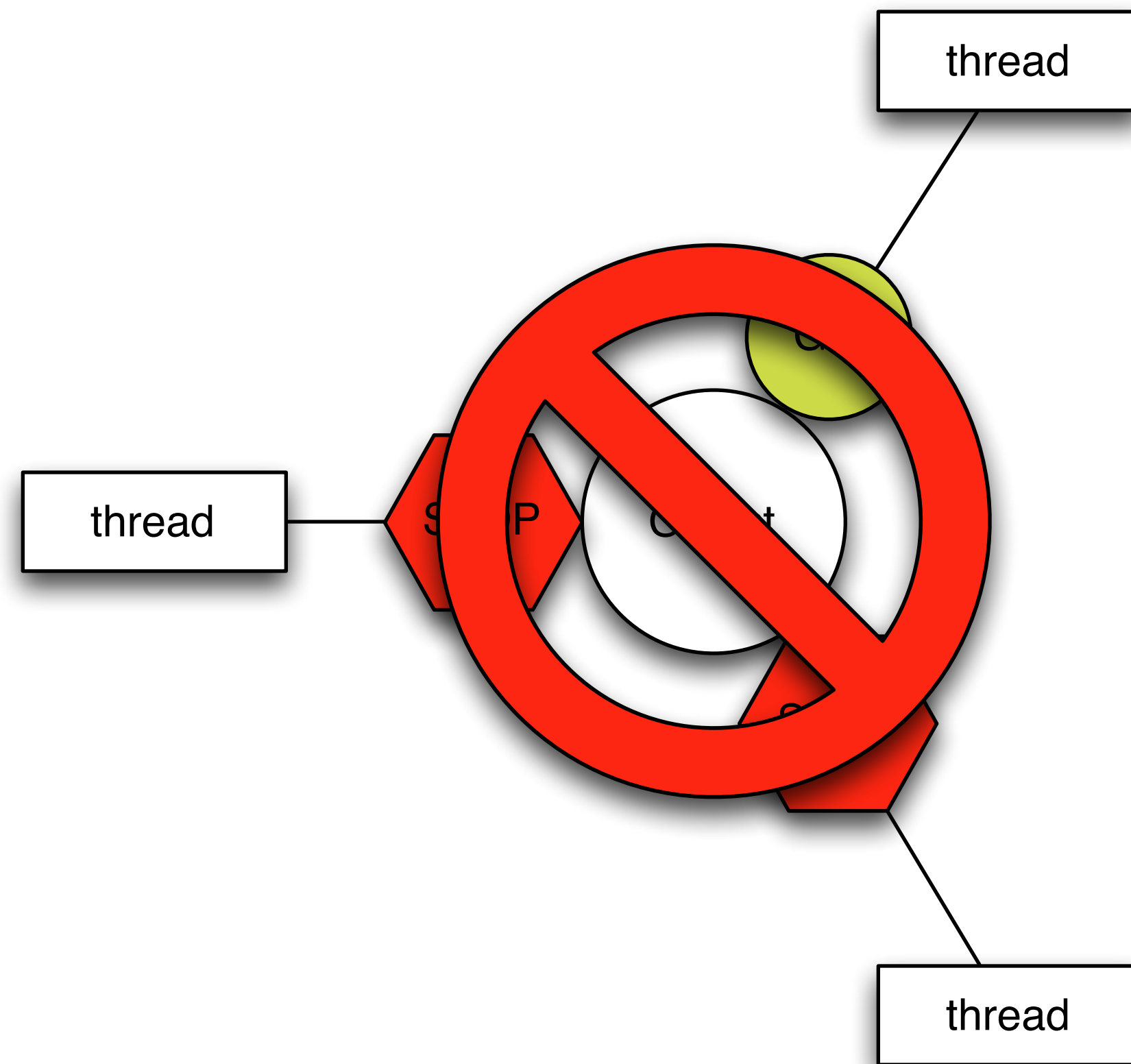
http://en.wikipedia.org/wiki/Amdahl%27s_law

Amdahl's Law

Each line represents 10% more
synchronization



Why No Raw Synchronization Primitives?





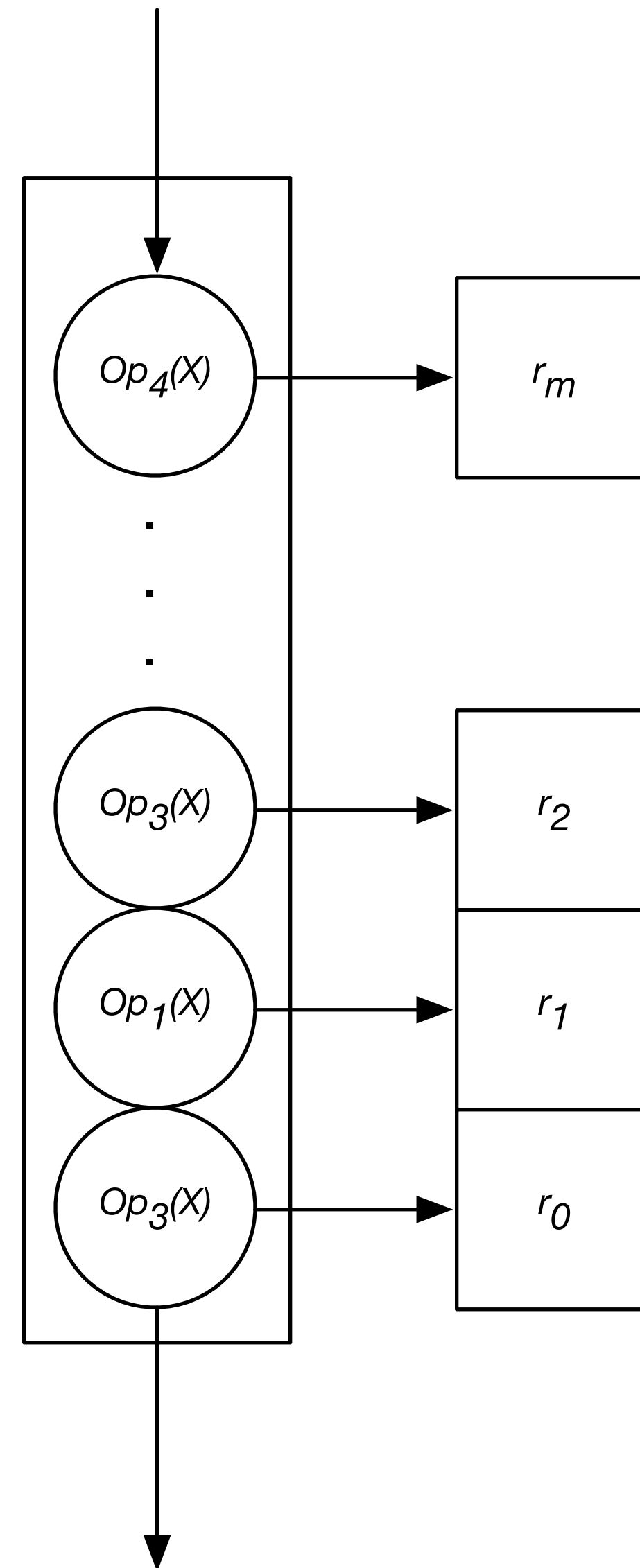
Mutexes and Sequential Consistency

```
class registry {  
    mutex _mutex;  
    unordered_map<string, string> _map;  
public:  
    void set(string key, string value) {  
        unique_lock<mutex> lock(mutex);  
        _map.emplace(move(key), move(value));  
    }  
  
    auto get(const string& key) -> string {  
        unique_lock<mutex> lock(mutex);  
        return _map.at(key);  
    }  
};
```

“It can be shown that programs that correctly use mutexes and `memory_order_seq_cst` operations to prevent all data races and use no other synchronization operations behave as if the operations executed by their constituent threads were simply interleaved, with each value computation of an object being taken from the last side effect on that object in that interleaving. This is normally referred to as ‘*sequential consistency*.’”

– C++11 Standard 1.10.21

Mutexes and Sequential Consistency



Mutexes and Sequential Consistency

- A mutex serializes a set of operations, Op_n , where the operation is the code executed while the mutex is locked
- Operations are interleaved and may be executed in any order and may be repeated
- Each operation takes an argument, X , which is the set of all objects mutated under all operations
 - X may not be safely read or written without holding the lock if it may be modified by a task holding the lock
- Each operation may yield a result, r_m , which can communicate information about the state of X while it's associated operation was executed
- The same is true of all atomic operations

Mutexes and Sequential Consistency

```
class registry {
    serial_queue _q;

    using map_t = unordered_map<string, string>;

    shared_ptr<map_t> _map = make_shared<map_t>();
public:
    void set(string key, string value) {
        _q.async([_map = _map](string key, string value) {
            _map->emplace(move(key), move(value));
        }, move(key), move(value));
    }

    auto get(string key) -> future<string> {
        return _q.async([_map = _map](string key) {
            return _map->at(key);
        }, move(key));
    }
};
```

Mutexes and Sequential Consistency

```
class registry {
    serial_queue _q;

    using map_t = unordered_map<string, string>;

    shared_ptr<map_t> _map = make_shared<map_t>();
public:
    void set(string key, string value) {
        _q.async([_map = _map](string key, string value) {
            _map->emplace(move(key), move(value));
        }, move(key), move(value));
    }

    auto get(string key) -> future<string> {
        return _q.async([_map = _map](string key) {
            return _map->at(key);
        }, move(key));
    }

    void set(vector<pair<string, string>> sequence) {
        _q.async([_map = _map](vector<pair<string, string>> sequence) {
            _map->insert(make_move_iterator(begin(sequence)), make_move_iterator(end(sequence)));
        }, move(sequence));
    }
};
```


Mutexes and Sequential Consistency

- The transformation mutex to serial queue places an upper-bound
 - Synchronization overhead
 - Time to issue operation

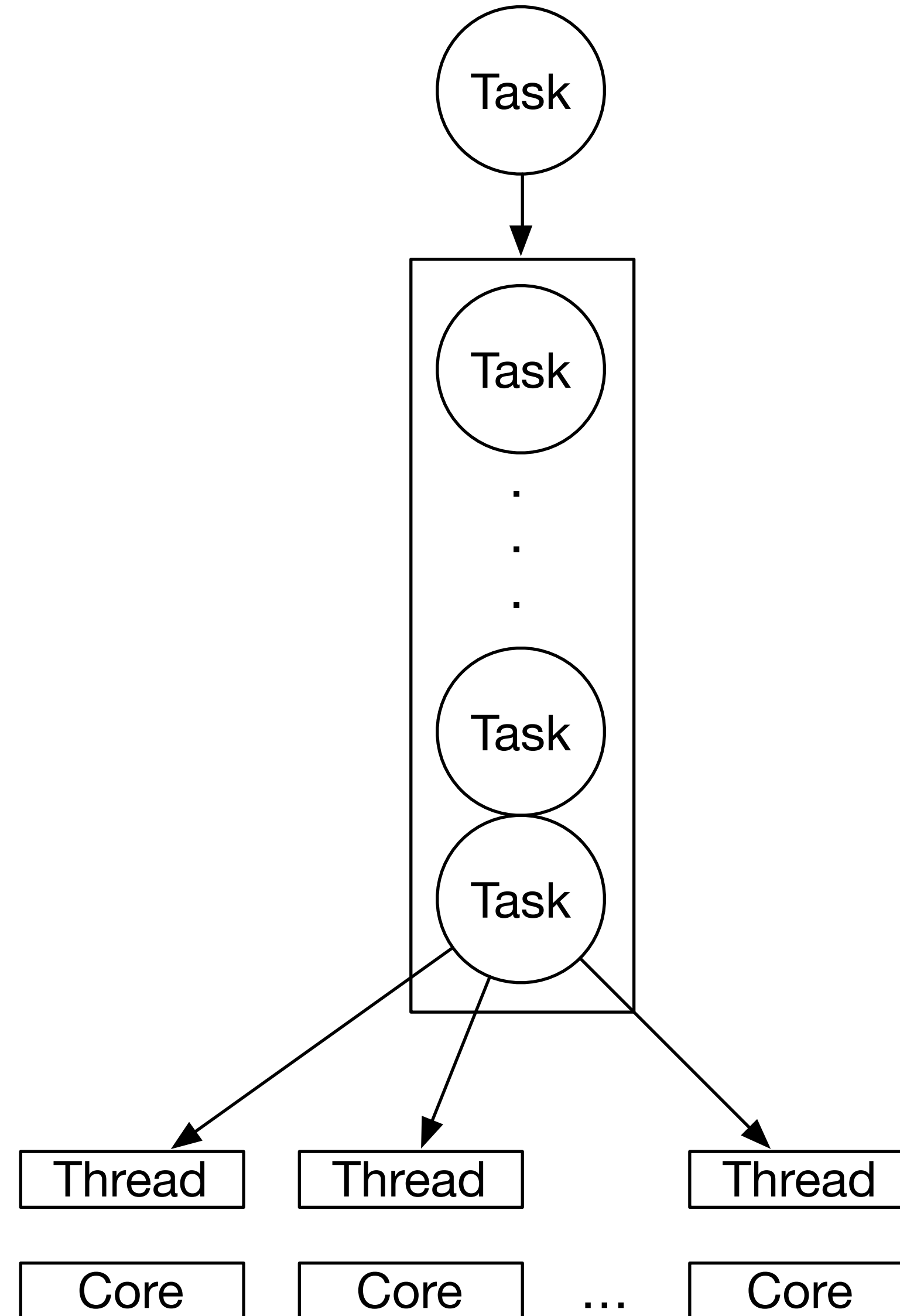
Threads and Tasks

- Thread: Execution environment consisting of a stack and processor state running in parallel to other threads
- Task: A unit of work, often a function, to be executed on a thread
- Tasks are scheduled on a thread pool to optimize machine utilization

- C++14 does not (really) have a task system
 - Threads
 - Futures
- It is implementation defined if `std::async()` spins up a thread or executes on a thread pool.

- Portable Reference Implementation in C++14
- Windows - Window Thread Pool and PPL
- Apple - Grand Central Dispatch (libdispatch)
 - open source, runs on Linux and Android
- Intel TBB - many platforms
 - open source
- HPX - many platforms
 - open source

Building a Task System



<http://docs.oracle.com/cd/E19253-01/816-5137/ggedn/index.html>

Building a Task System

```
using lock_t = unique_lock<mutex>;

class notification_queue {
    deque<function<void()>> _q;
    mutex _mutex;
    condition_variable _ready;

public:
    void pop(function<void()>& x) {
        lock_t lock{_mutex};
        while (_q.empty()) _ready.wait(lock);
        x = move(_q.front());
        _q.pop_front();
    }

    template<typename F>
    void push(F&& f) {
        {
            lock_t lock{_mutex};
            _q.emplace_back(forward<F>(f));
        }
        _ready.notify_one();
    }
};
```

Building a Task System

```
class task_system {
    const unsigned          _count{thread::hardware_concurrency()};
    vector<thread>          _threads;
    notification_queue       _q;

    void run(unsigned i) {
        while (true) {
            function<void()> f;
            _q.pop(f);
            f();
        }
    }

public:
    task_system() {
        for (unsigned n = 0; n != _count; ++n) {
            _threads.emplace_back([&, n]{ run(n); });
        }
    }

    ~task_system() {
        for (auto& e : _threads) e.join();
    }

    template <typename F>
    void async_(F&& f) {
        _q.push(forward<F>(f));
    }
};
```

Building a Task System

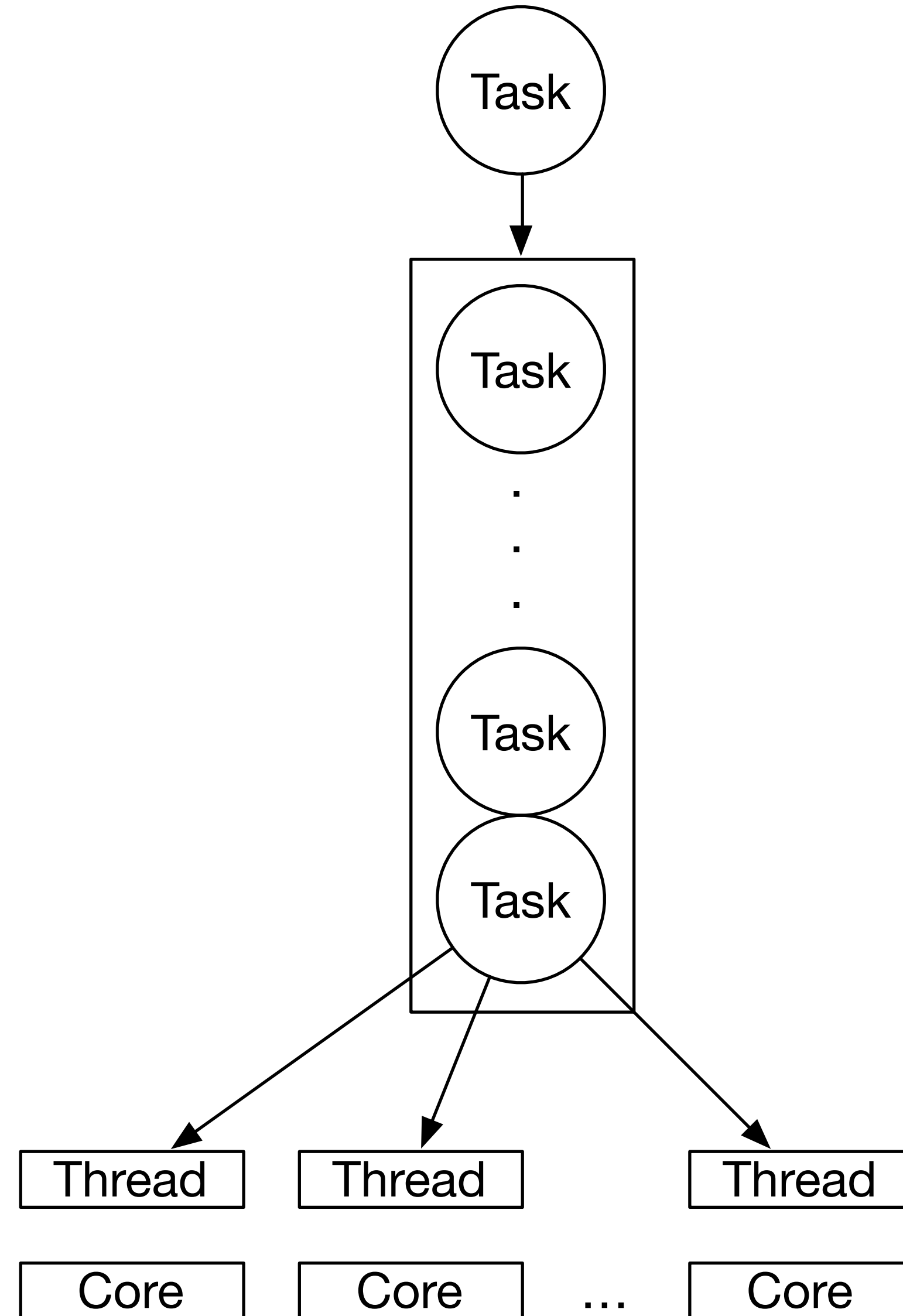
```
class notification_queue {
    deque<function<void()>> _q;
    bool _done{false};
    mutex _mutex;
    condition_variable _ready;

public:
    void done() {
        {
            unique_lock<mutex> lock{_mutex};
            _done = true;
        }
        _ready.notify_all();
    }

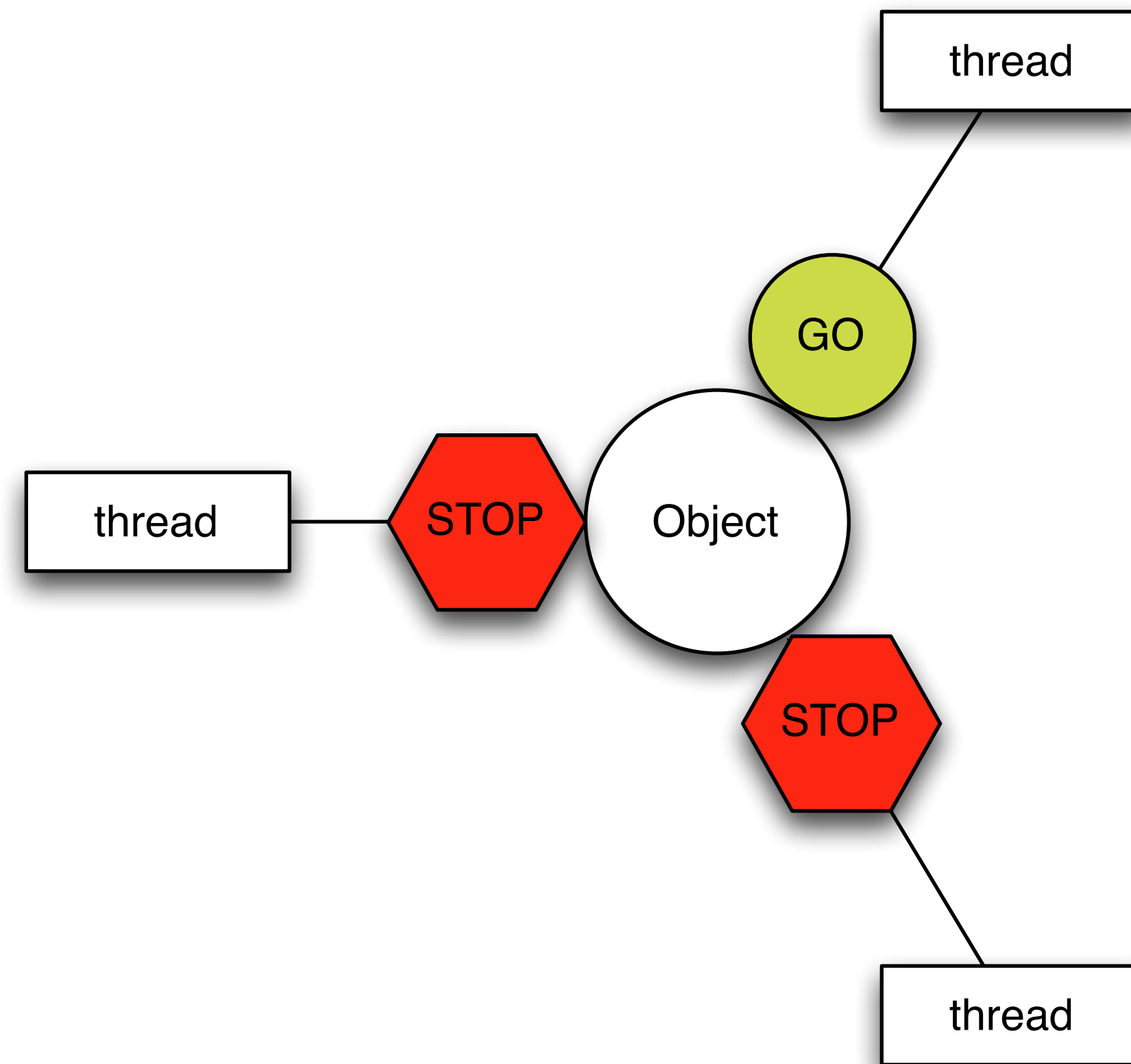
    bool pop(function<void()>& x) {
        lock_t lock{_mutex};
        while (_q.empty() && !_done) _ready.wait(lock);
        if (_q.empty()) return false;
        x = move(_q.front());
        _q.pop_front();
        return true;
    }

    template<typename F>
    void push(F&& f) {
        {
            lock_t lock{_mutex};
            q.emplace back(forward<F>(f));
        }
    }
};
```

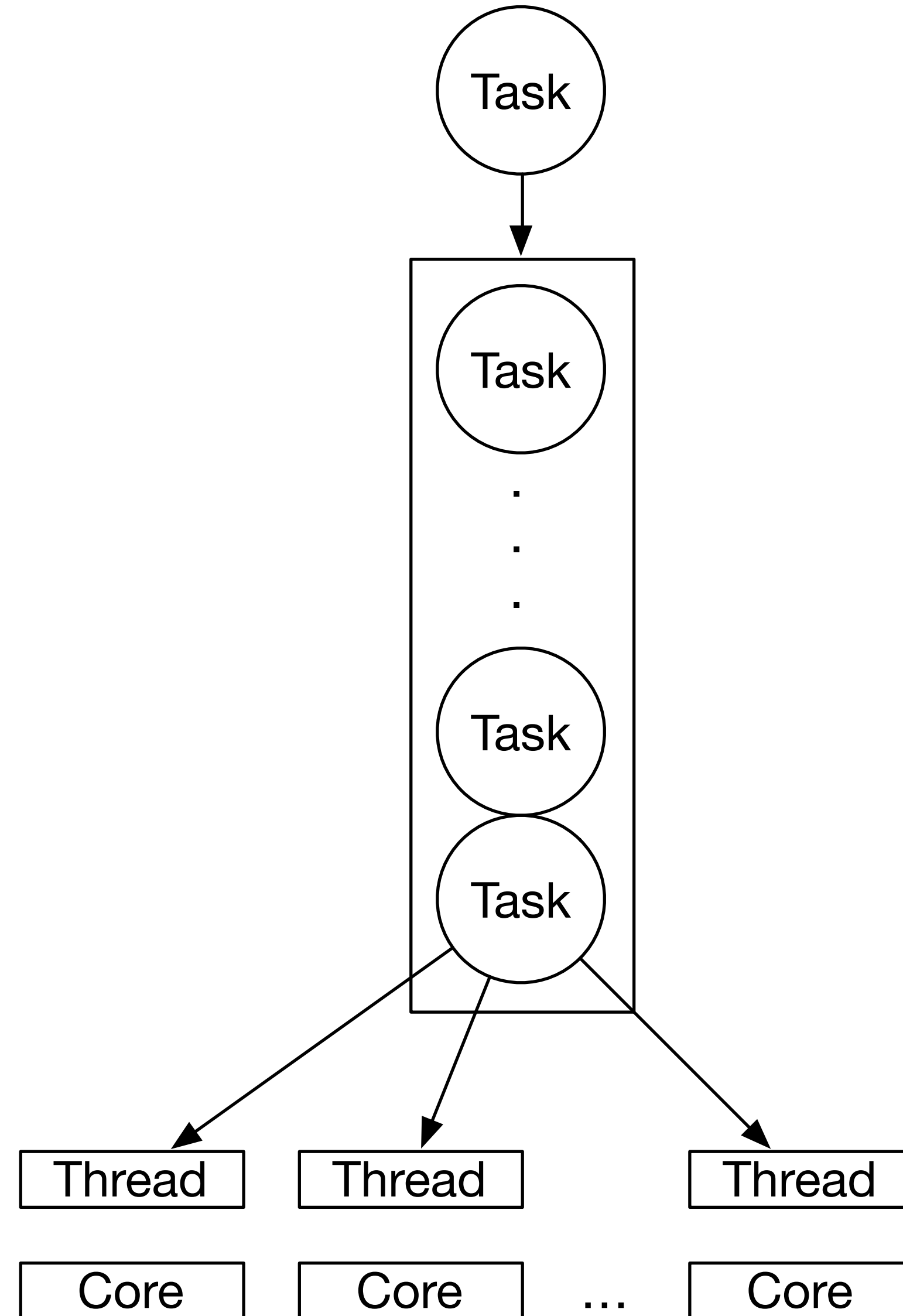

Building a Task System



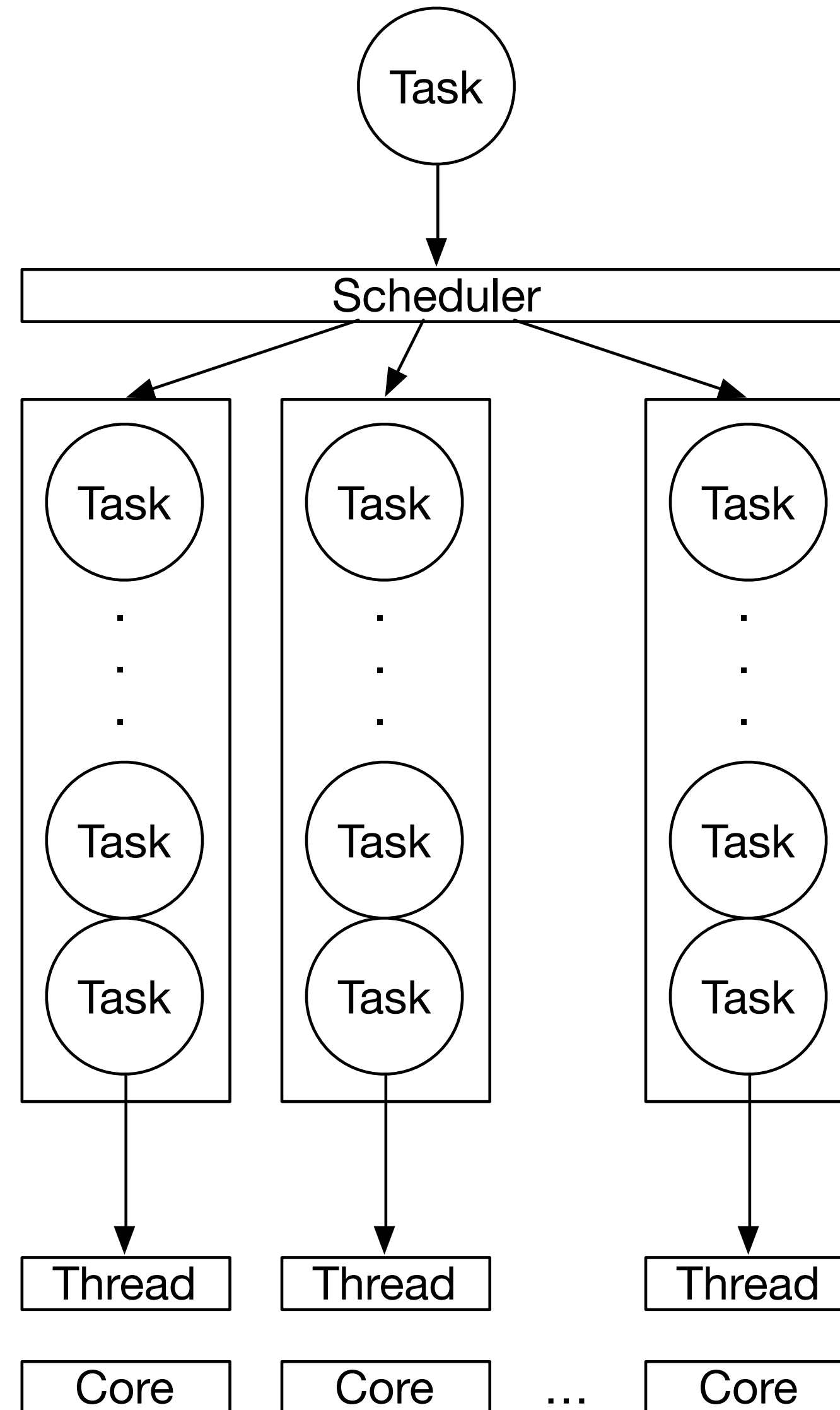
Why No Raw Synchronization Primitives?



Building a Task System



Building a Task System



Building a Task System

```
class task_system {
    const unsigned                _count{thread::hardware_concurrency()};
    vector<thread>                _threads;
    vector<notification_queue>    _q{_count};
    atomic<unsigned>              _index{0};

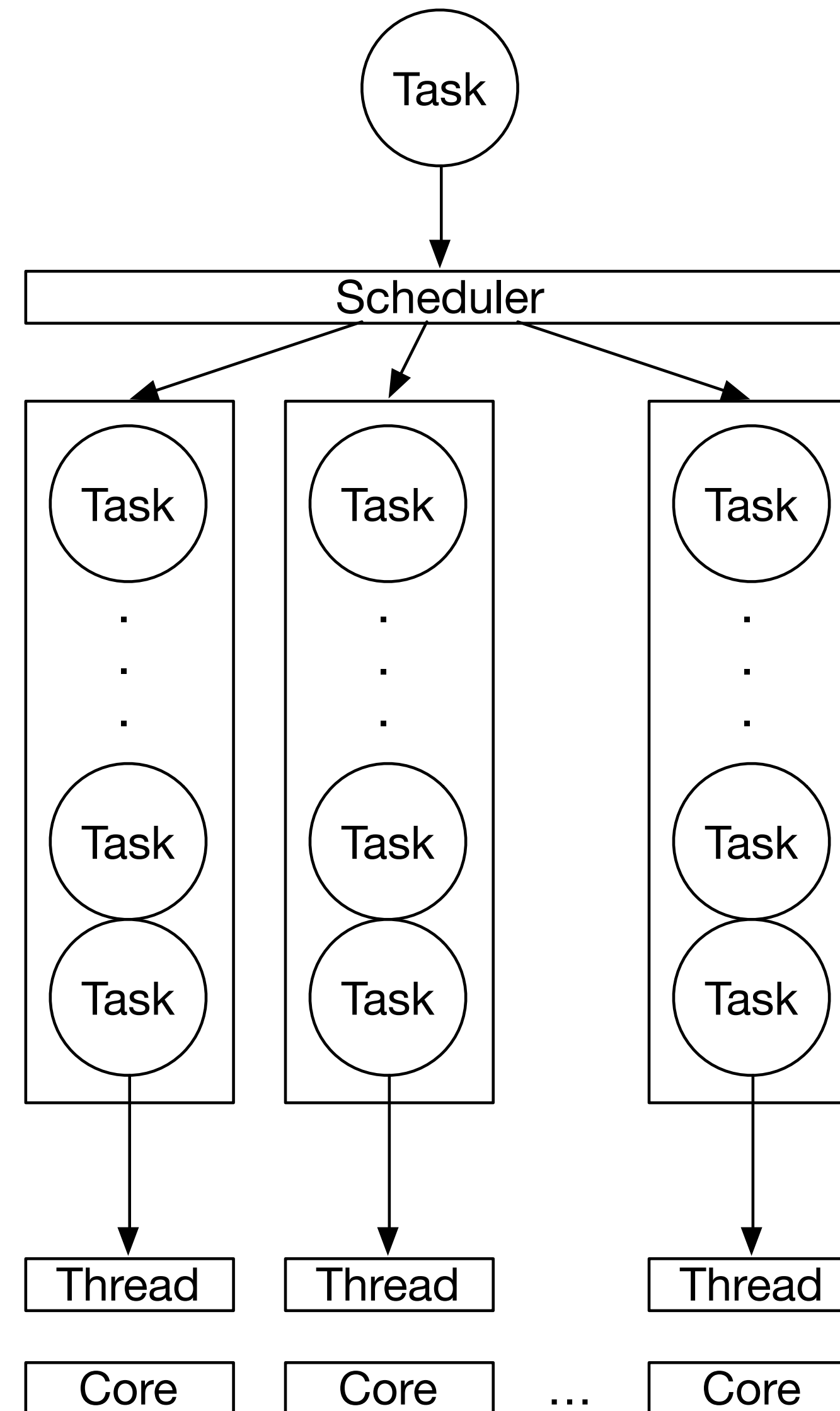
    void run(unsigned i) {
        while (true) {
            function<void()> f;
            if (!_q[i].pop(f)) break;
            f();
        }
    }

public:
    task_system() { ... }

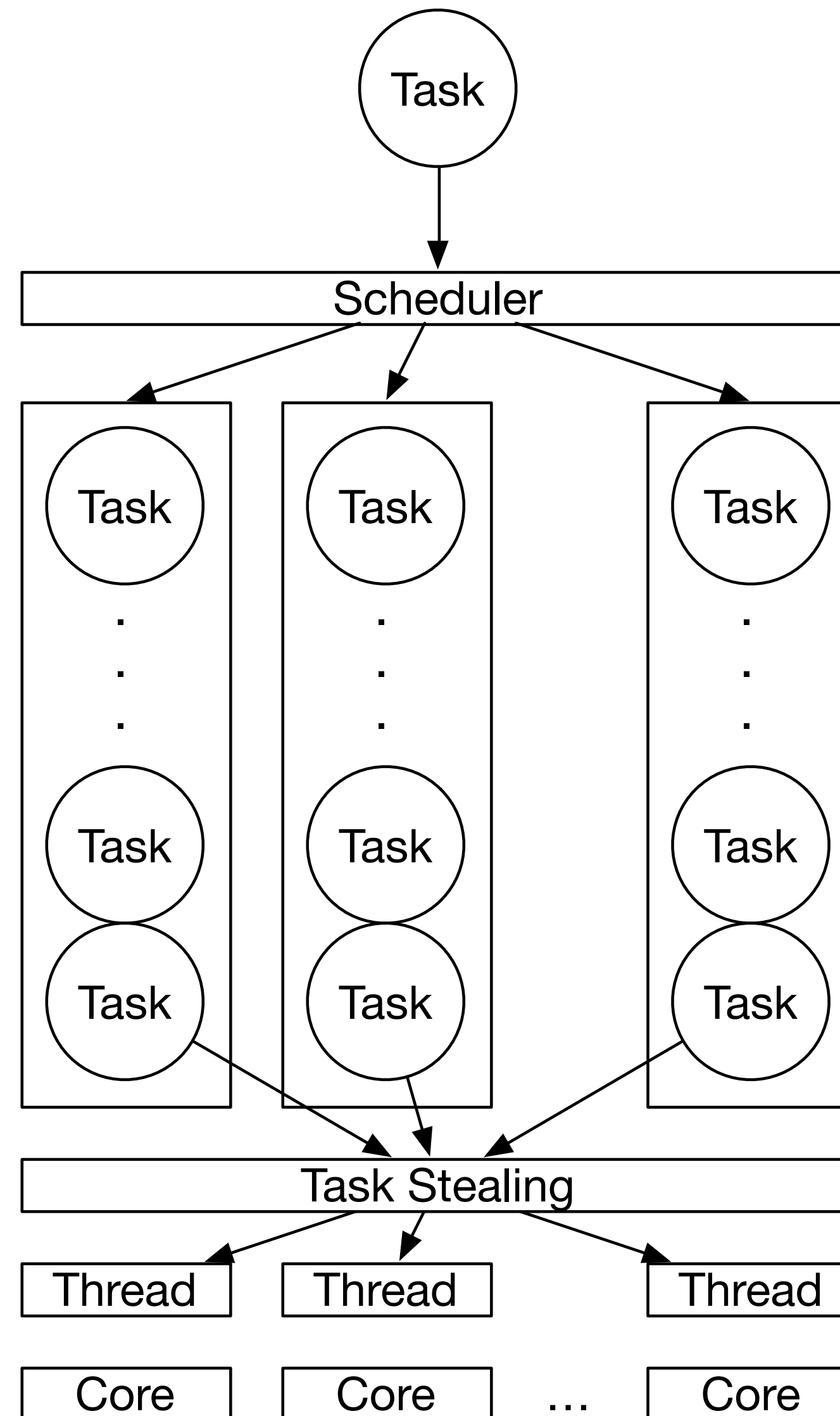
    ~task_system() {
        for (auto& e : _q) e.done();
        for (auto& e : _threads) e.join();
    }

    template <typename F>
    void async_(F&& f) {
        auto i = _index++;
        _q[i % _count].push(forward<F>(f));
    }
};
```


Building a Task System



Building a Task System



Building a Task System

```
class notification_queue {
    deque<function<void()>> _q;
    bool _done{false};
    mutex _mutex;
    condition_variable _ready;

public:
    bool try_pop(function<void()>& x) {
        lock_t lock{_mutex, try_to_lock};
        if (!lock || _q.empty()) return false;
        x = move(_q.front());
        _q.pop_front();
        return true;
    }

    template<typename F>
    bool try_push(F&& f) {
        {
            lock_t lock{_mutex, try_to_lock};
            if (!lock) return false;
            _q.emplace_back(forward<F>(f));
        }
        _ready.notify_one();
        return true;
    }

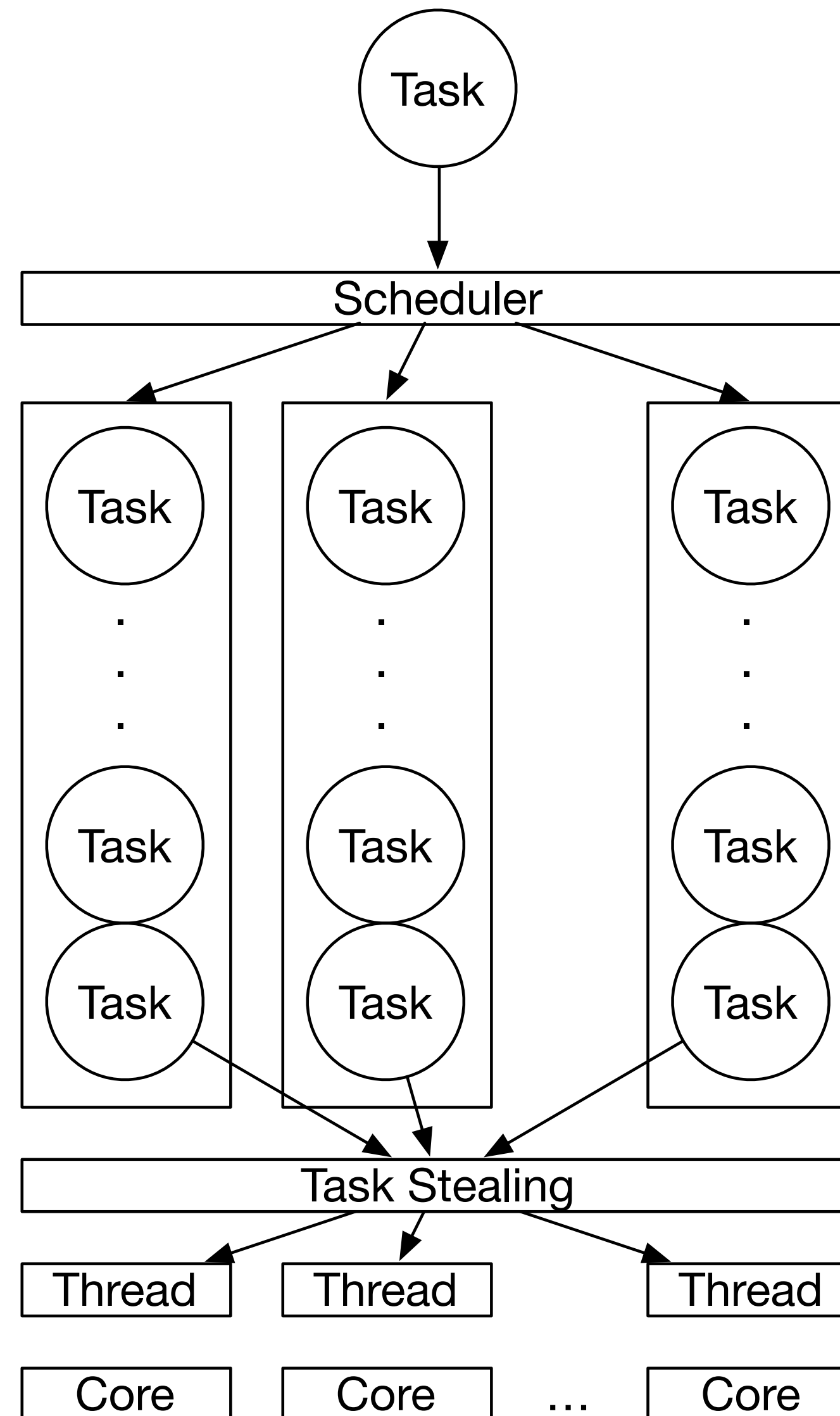
    void done() {
        {
            unique lock<mutex> lock{ mutex};

```

Building a Task System

```
void run(unsigned i) {  
    while (true) {  
        function<void()> f;  
  
        for (unsigned n = 0; n != _count; ++n) {  
            if (_q[(i + n) % _count].try_pop(f)) break;  
        }  
        if (!f && !_q[i].pop(f)) break;  
  
        f();  
    }  
}  
  
public:  
    task_system() { ... }  
  
    ~task_system() { ... }  
  
    template <typename F>  
    void async_(F&& f) {  
        auto i = _index++;  
  
        for (unsigned n = 0; n != _count * K; ++n) {  
            if (_q[(i + n) % _count].try_push(forward<F>(f))) return;  
        }  
  
        _q[i % _count].push(forward<F>(f));  
    }  
};
```

Building a Task System



- Compared to Apple's Grand Central Dispatch (libdispatch)



C++14 compatible async with libdispatch

```
template <class Function, class... Args>
auto async(Function&& f, Args&&... args )
{
    using result_type = std::result_of_t<std::decay_t<Function>(std::decay_t<Args>...)>;
    using packaged_type = std::packaged_task<result_type>;

    auto _p = new packaged_type(std::bind([&f = std::forward<Function>(f)](Args&... args) {
        return _f(std::move(args)...);
    }, std::forward<Args>(args)...));

    auto result = _p->get_future();

    dispatch_async_f(dispatch_get_global_queue(DISPATCH_QUEUE_PRIORITY_DEFAULT, 0),
        _p, [](void* p) {
            auto _p = static_cast<packaged_type*>(p);
            (*_p)();
            delete _p;
        });

    return result;
}
```


- Written with ASIO (Boost 1.62.0)

```
class task_system {
    io_service          _service;
    vector<thread>      _threads;
    unique_ptr<io_service::work> _work{make_unique<io_service::work>(_service)};

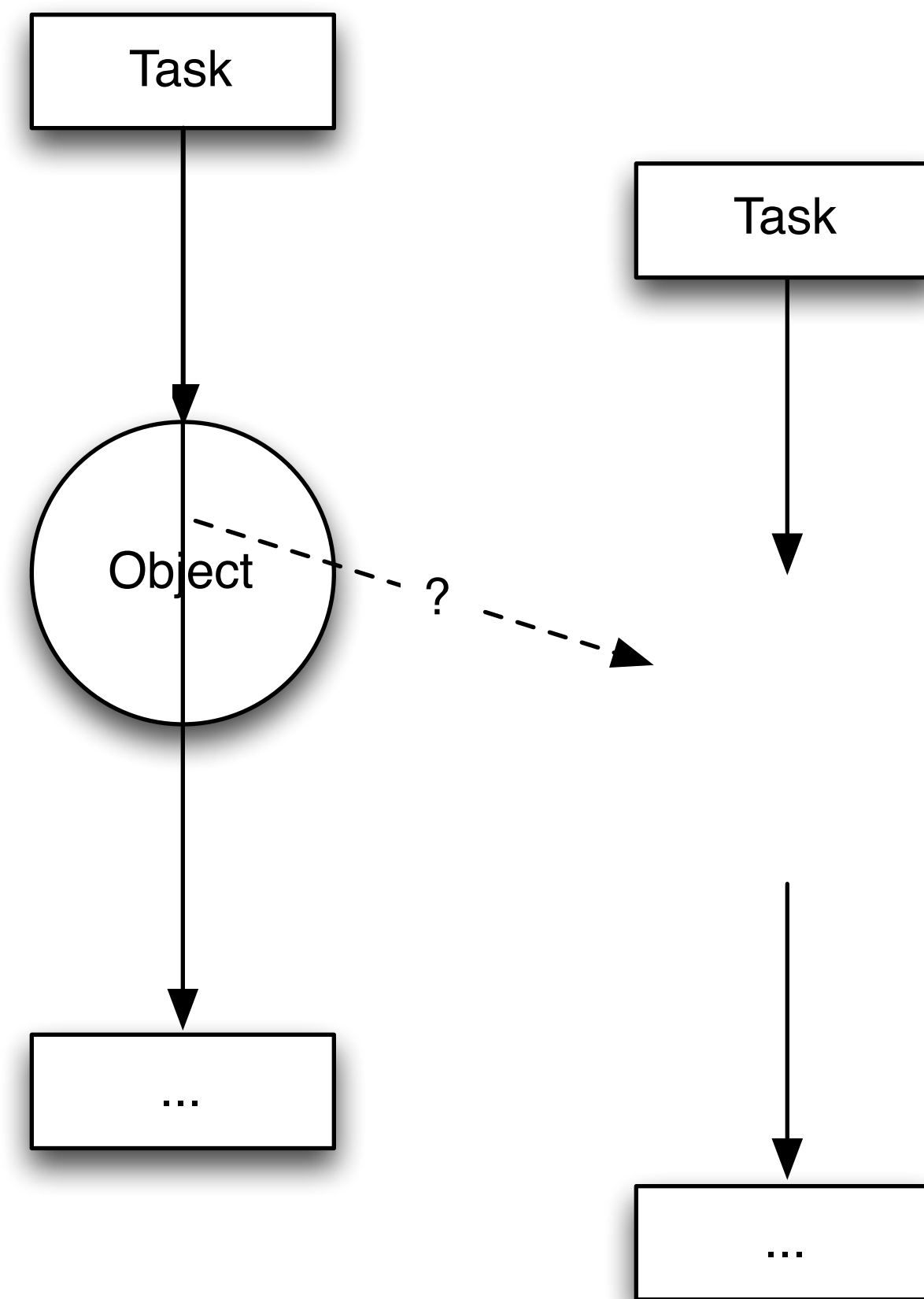
public:
    task_system() {
        for (unsigned n = 0; n != thread::hardware_concurrency(); ++n) {
            _threads.emplace_back( [&]{
                _service.run();
            });
        }
    }

    ~task_system() {
        _work.reset();
        for (auto& e : _threads) e.join();
    }

    template <typename F>
    void async_(F&& f) {
        _service.post(forward<F>(f));
    }
};
```



No Raw Synchronization Primitives



```
future<cpp_int> x = async([]{ return fibonacci<cpp_int>(1'000'000); });  
  
// Do Something  
  
cout << x.get() << endl;
```

- Fibonacci is often used as an example for parallel algorithms
 - Please stop...

Public Service Announcement - How to Write Fibonacci

```
template <typename T, typename N, typename Op>
T power(T x, N n, Op op)
{
    if (n == 0) return identity_element(op);

    while ((n & 1) != 0) {
        n >>= 1;
        x = op(x, x);
    }

    T result = x;
    n >>= 1;
    while (n != 0) {
        x = op(x, x);
        if ((n & 1) != 0) result = op(result, x);
        n >>= 1;
    }
    return result;
}
```

Egyptian Multiplication (Russian Peasant Algorithm)

See “From Mathematics to Generic Programming” - Alex Stepanov and Dan Rose

Public Service Announcement - How to Write Fibonacci

```
template <typename N>
struct multiply_2x2 {
    array<N, 4> operator()(const array<N, 4>& x, const array<N, 4>& y)
    {
        return { x[0] * y[0] + x[1] * y[2], x[0] * y[1] + x[1] * y[3],
                 x[2] * y[0] + x[3] * y[2], x[2] * y[1] + x[3] * y[3] };
    }
};
```

```
template <typename N>
array<N, 4> identity_element(const multiply_2x2<N>&) { return { N(1), N(0), N(0), N(1) }; }
```

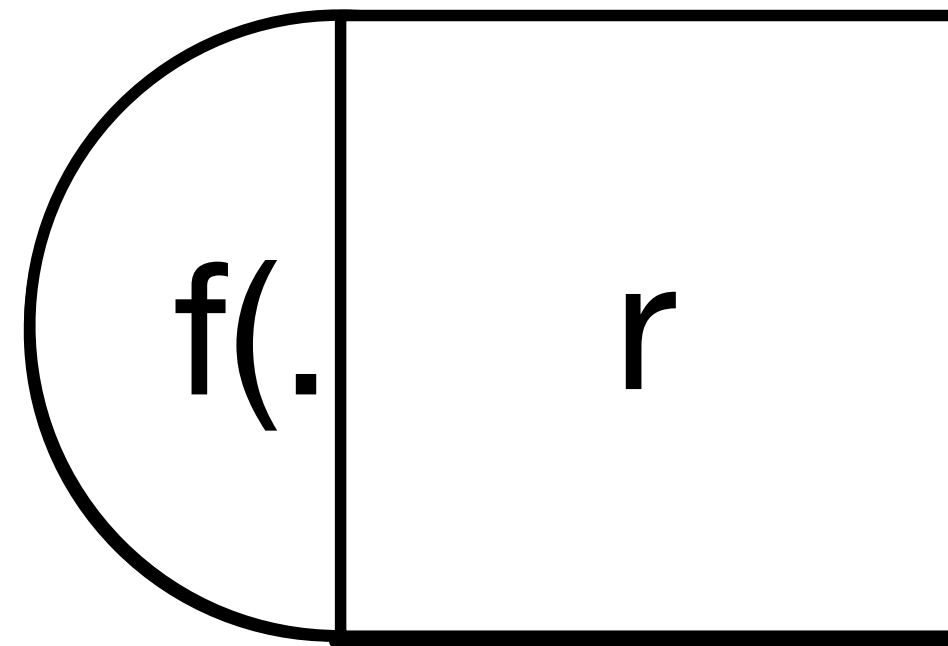
```
template <typename R, typename N>
R fibonacci(N n) {
    if (n == 0) return R(0);
    return power(array<R, 4>{ 1, 1, 1, 0 }, N(n - 1), multiply_2x2<R>())[0];
}
```

$$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^n = \begin{bmatrix} F_{n+1} & F_n \\ F_n & F_{n-1} \end{bmatrix}$$

Futures

```
future<cpp_int> x = async([]{ return fibonacci<cpp_int>(1'000'000); });  
  
// Do Something  
  
cout << x.get() << endl;
```


19532821287077577316320149475962563324435429965918733969534051945716252578870156947666419876341501461288795
24335220236084625510912019560233744015438115196636156919962125642894303370113827800638002767411527927466669
86557837931882283206127149758323033485489348957259923072291290192820926433162752173086146001791258204269965
99360209593392020051848620284024473431398113674187202038684801753185386211128781082406177413832935545616876
06454065125954718029126547942894036981659206361019359291352135410376799082940320155702716115395031975973247
78216295763162965335669477766328506234524559346064757502593581344345781676764625878859011372729907372947851
14480895724561915035070255895291168685500088020132334587472177947814475467920160901706425856293597475465327
57575740077432034913428785189795354304734560307765078938767286539166799232817449361991523768149557632085371
04785970618843873153058239562756087906310781900497516959470973671389174570455520213512335079440336071203050
41446852210415650373210679322756258647511914611417360349681217380234224786080292021093192496490409832397066
83247054441763512526732455275419501683845206023007394959854279298297831204382115757645787692495583351402522
15272066244180900325938075362849179668095297118507191379833678873770459913639333955814212036990261617972113
25091840023055327607104316478190974300434647793363287601469996128023925829471557316688943339455429292871877
48774789204296166356536610796023919702109728472966709427334586344798048633944635211654971507261342768205479
32093175079888010130416027982506354182344034558742236701282666356934611294613123128389060036547327660245693
15151850018328483150645480029978935985161237074046158229354440701748339514575869547491750264542126364262224
72060048855462589961190475892101224280542898621594646662478564373572217775549876087685912030118551635668902
01034463998397732663888903650784161807091545252992759735213957415477729146008794314339156060445825107823511
6627189263792331301464388059787946844487906057670829840989627426663569682474293386740207436559426057944790
71193052258931590719386545525880429139747140181849169733838138446154843063123649290835584278078456131936457
55911722136946338180311600307896211668652895953778436464402382516362449718197385444149563131714002850338928
22274134603018094224837216321854717270452813824078425638747365249141118080783866506339945376239206700513391
87333107136069698189628284763245423299306272870457991293245741167533902274499963096566680922262516468582544
55785134982414412726124015815753818098466667145006988839178551800894370189025721992485208742915560261917752
28124660628996787166529678487268484905041328497297712688011639978376434280202452251550102240354169885185375
01584673881194047619720619603126534496759917893244478170702904446589571950228809157793897642423751814020998
99958161231477902295781100168670186738619861797138139854666281969548553740707356228616165539428076418408092
12047932816683005984504787929406356318097479755152035094682765918741610907637506902765294367561539803261388
00104485041004520227541880045735620705421800662063441346306055080001375010053760250440113617210176881147264



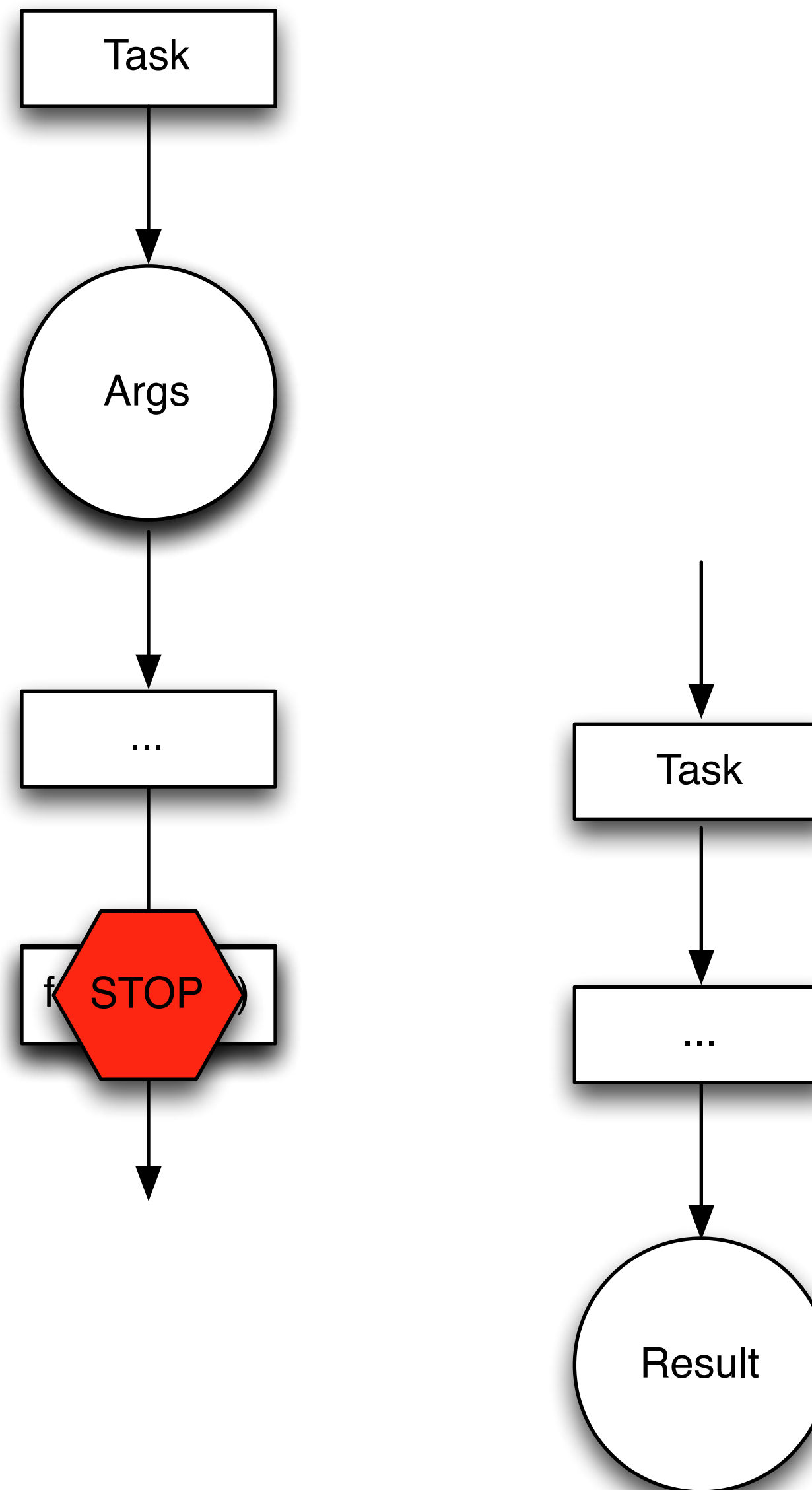
- Futures allow minimal code transformations to express dependencies

Exception Marshalling

```
future<cpp_int> x = async([]{  
    throw runtime_error("failure");  
    return fibonacci<cpp_int>(1'000'000);  
});  
  
// Do Something  
  
try {  
    cout << x.get() << endl;  
} catch (const runtime_error& error) {  
    cout << error.what() << endl;  
}
```

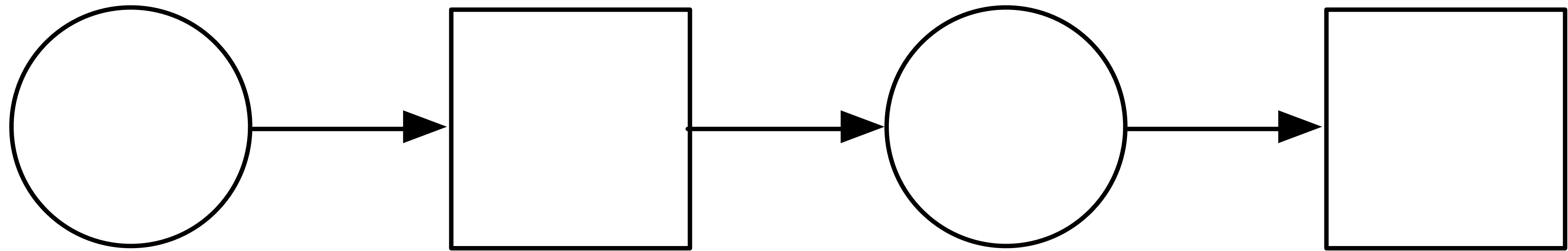
failure

No Raw Synchronization Primitives



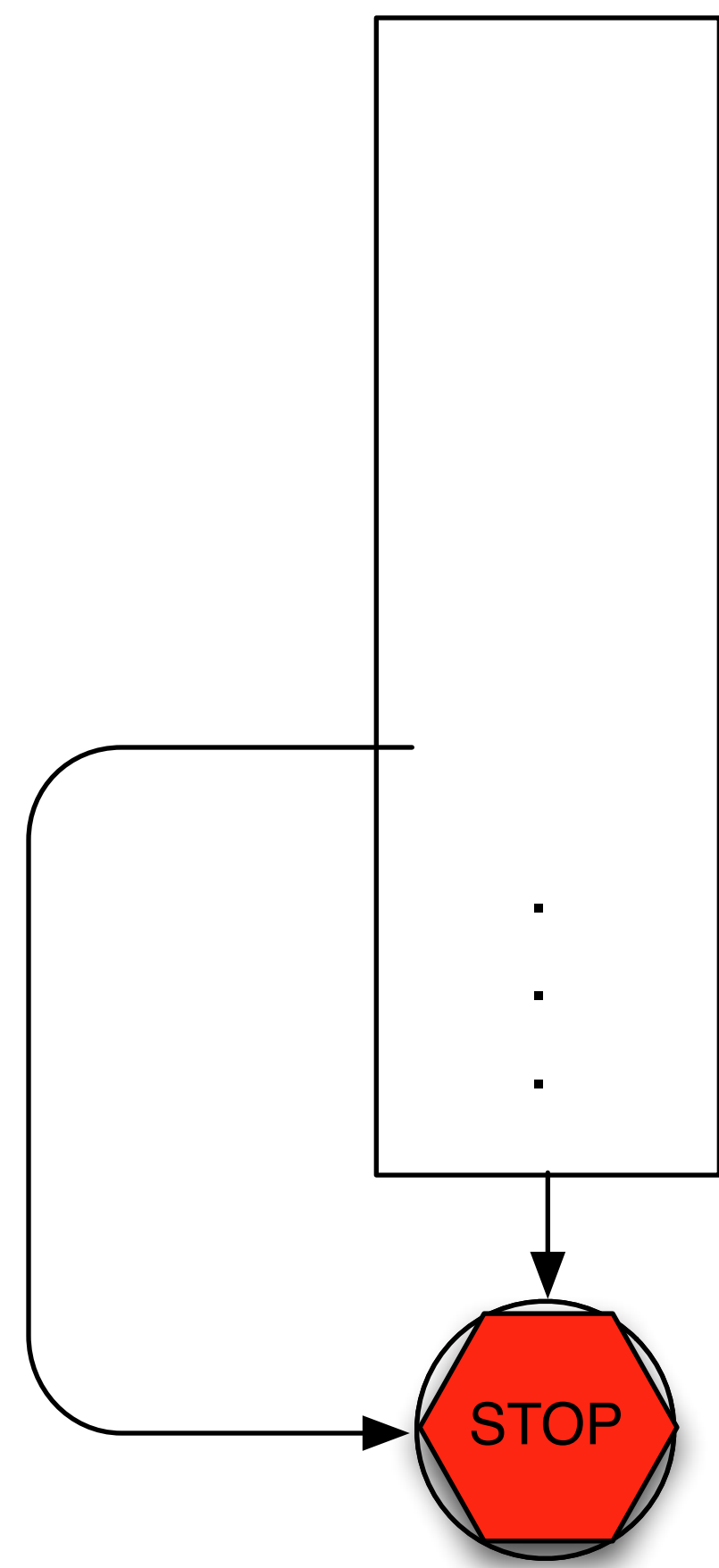
Futures: What year is this?

- C++11 (through 17) futures lack:
 - Continuations - `.then()`
 - Joins - `when_all()`
 - Split
 - Cancellation
 - Progress Monitoring (Except Ready)
- And C++11 futures don't compose (easily) to add these features

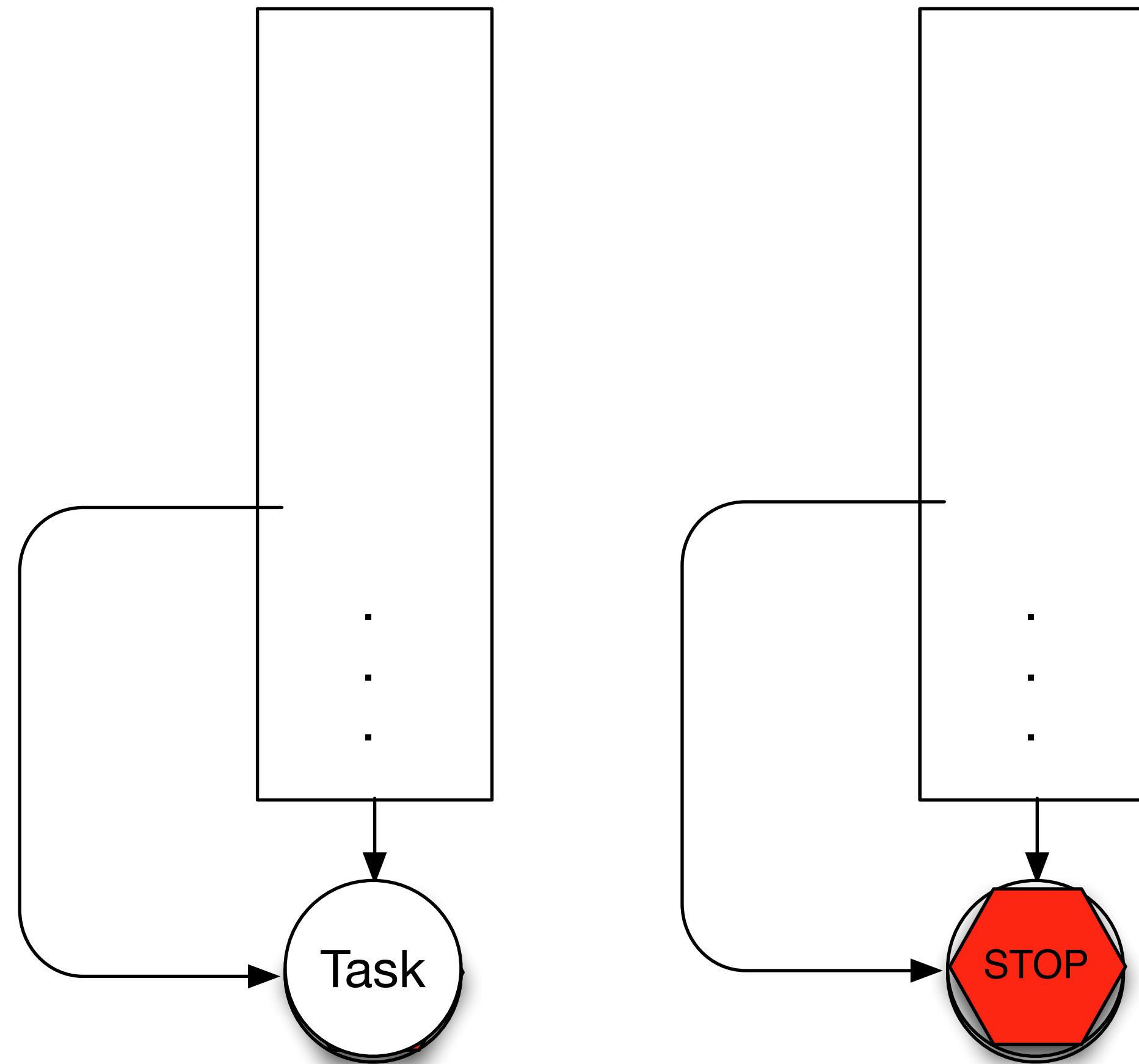


- Blocking on `std::future.get()` has two problems
 - One thread resource is consumed, increasing contention
 - Possibly causing a deadlock in our tasking system!
 - Any subsequent non-dependent calculations on the task are also blocked
- C++17 doesn't have continuations
 - GCD has serialized queues and groups
 - PPL has chained tasks
 - TBB has flow graphs
 - TS Concurrency has `.then()`
 - Boost futures have them now

Futures: get() deadlock



Futures: get() deadlock



- Blocking on `std::future.get()`
 - Very difficult to use safely with a thread pool
 - C++11 allows `std::async()` to use a thread pool
 - But the futures must contain a task stealing `get()` operation to make this work
 - You cannot create an `std::future()` with this attribute except with `std::async()`
- Not just `get()` - *any* conditional blocking (condition variables, wait, ...) is problematic with a task system

Do not call `std::future.get()` or `std::future.wait()` when the originating task, or any subordinate task, is on the same queue, even if it is a concurrent queue

Important: You should never call the `dispatch_sync` or `dispatch_sync_f` function from a task that is executing in the same queue that you are planning to pass to the function. This is particularly important for serial queues, which are guaranteed to deadlock, but should also be avoided for concurrent queues.

<https://developer.apple.com/library/content/documentation/General/Conceptual/ConcurrencyProgrammingGuide/OperationQueues/OperationQueues.html>

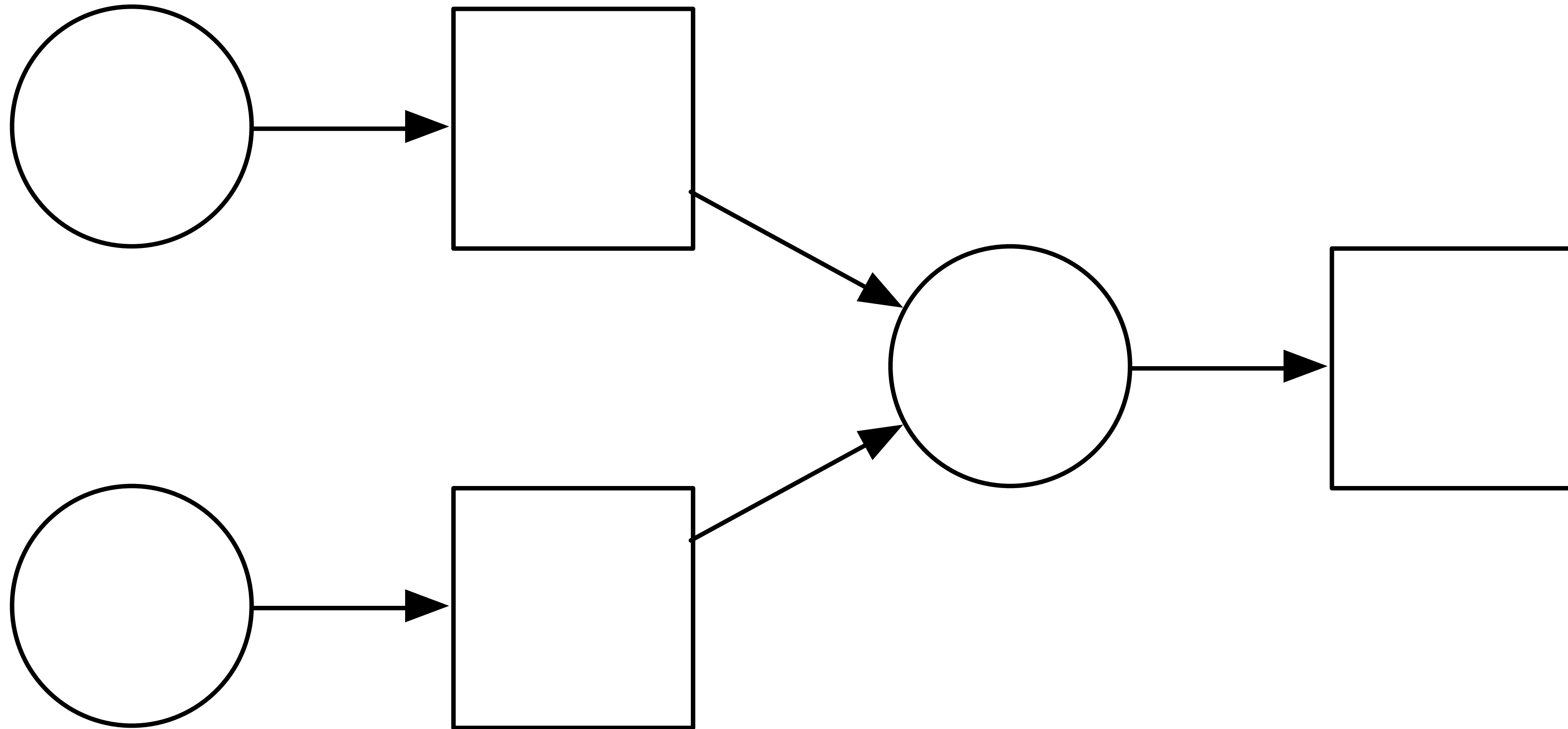
Futures: Continuations

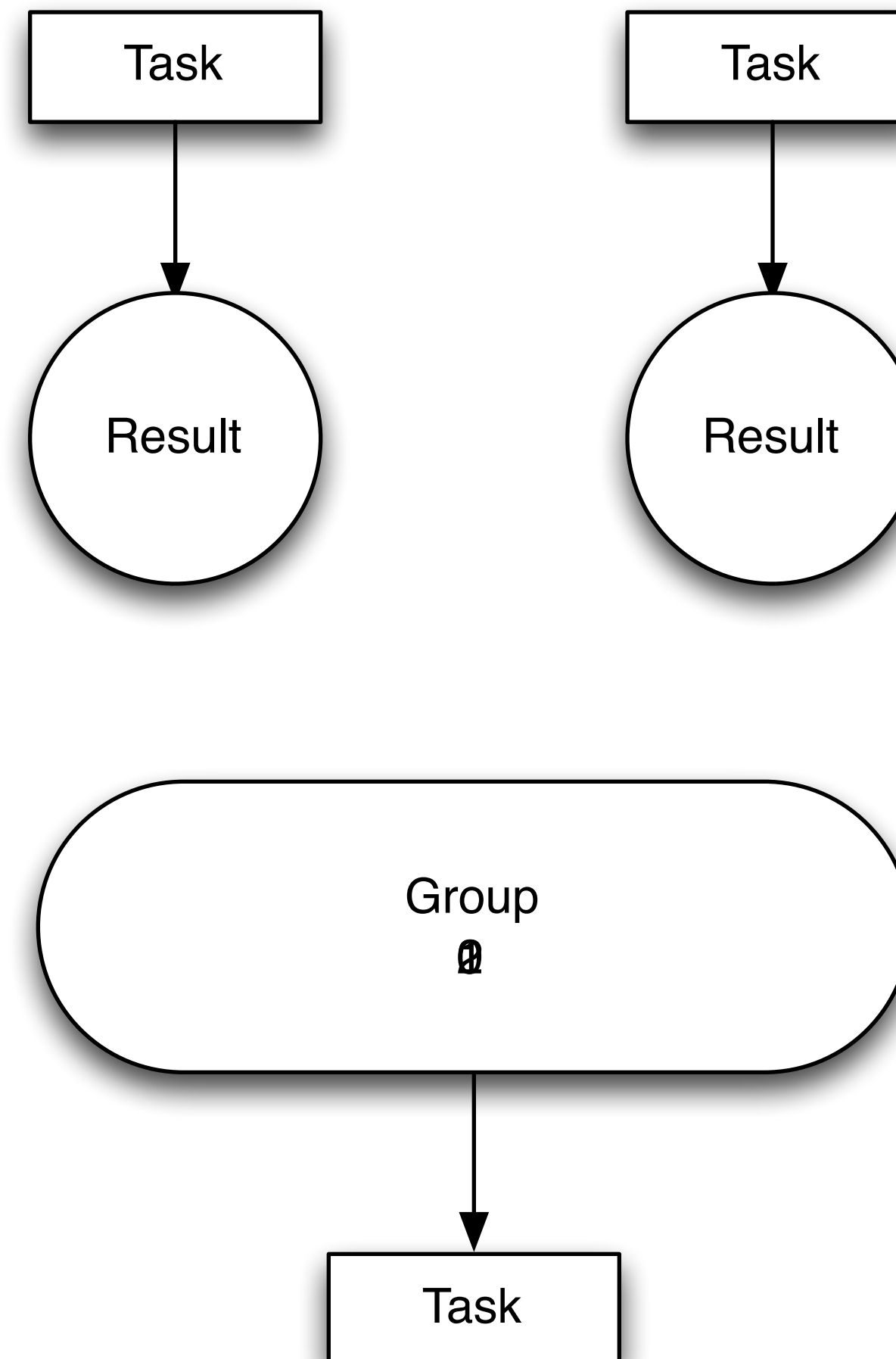
```
future<cpp_int> x = async([]{ return fibonacci<cpp_int>(1'000); });  
  
future<void> y = x.then([](future<cpp_int> x){ cout << x.get() << endl; });  
  
// Do something  
  
y.wait();
```

43466557686937456435688527675040625802564660517371780402481729089536555417949051890403879840079255169295922593080322634775209
689623239873322471161642996440906533187938298969649928516003704476137795166849228875

Futures vs Completion Handlers

- Completion handlers are callbacks, they must be known prior to the call
 - No need to synchronize between invoking and setting the continuation
- Futures allow setting the continuation after the sending call is in flight
 - Simpler to compose
 - Require synchronization between invoking and setting the continuation





Futures: Continuations

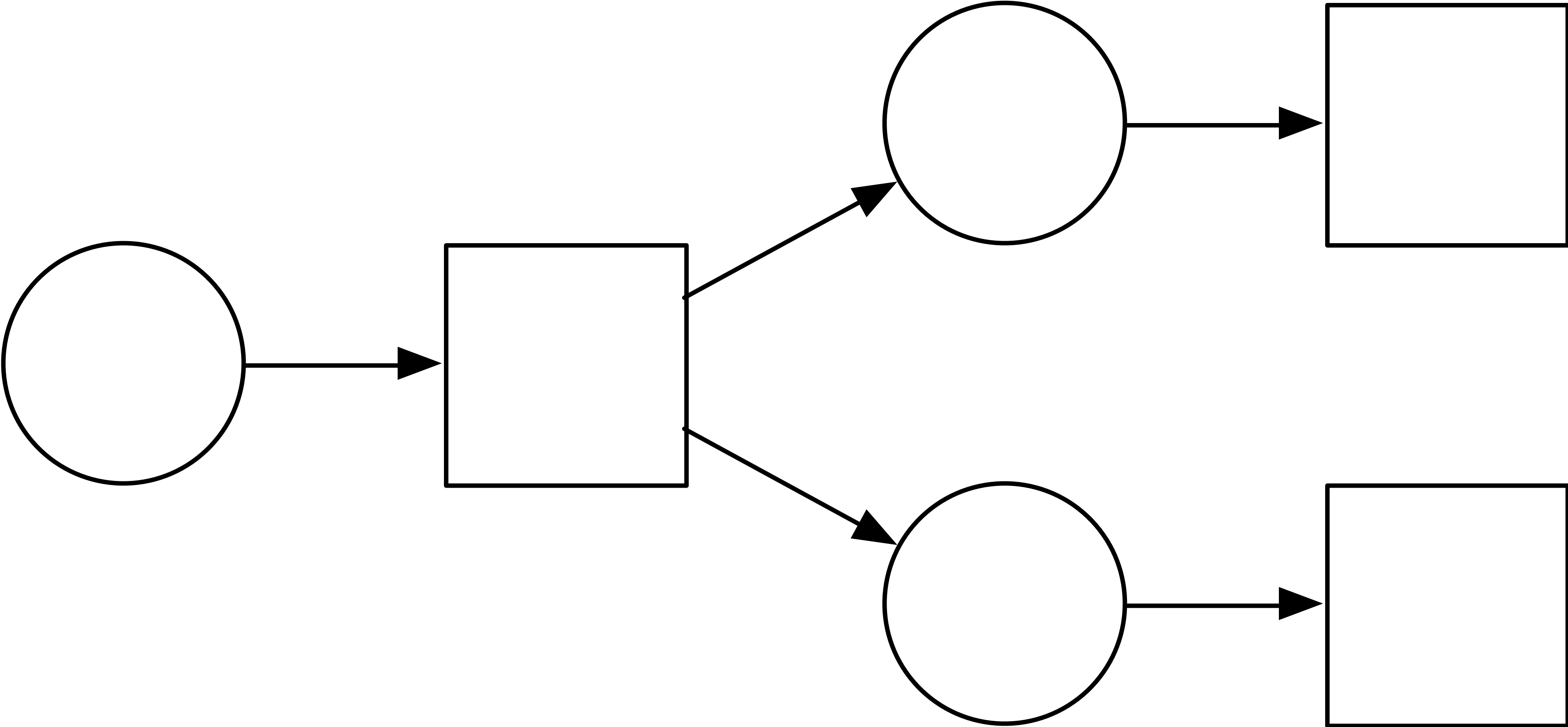
```
auto x = async([]{ return fibonacci<cpp_int>(1'000'000); });
auto y = async([]{ return fibonacci<cpp_int>(2'000'000); });

auto z = when_all(std::move(x), std::move(y)).then([](auto f){
    auto t = f.get();
    return cpp_int(get<0>(t).get() * get<1>(t).get());
});

cout << z.get() << endl;
```

f is a future tuple of futures

result is 626,964 digits



Futures: Continuations

```
future<cpp_int> x = async([]{ return fibonacci<cpp_int>(100); });
```

```
future<cpp_int> y = x.then([](future<cpp_int> x){ return cpp_int(x.get() * 2); });
```

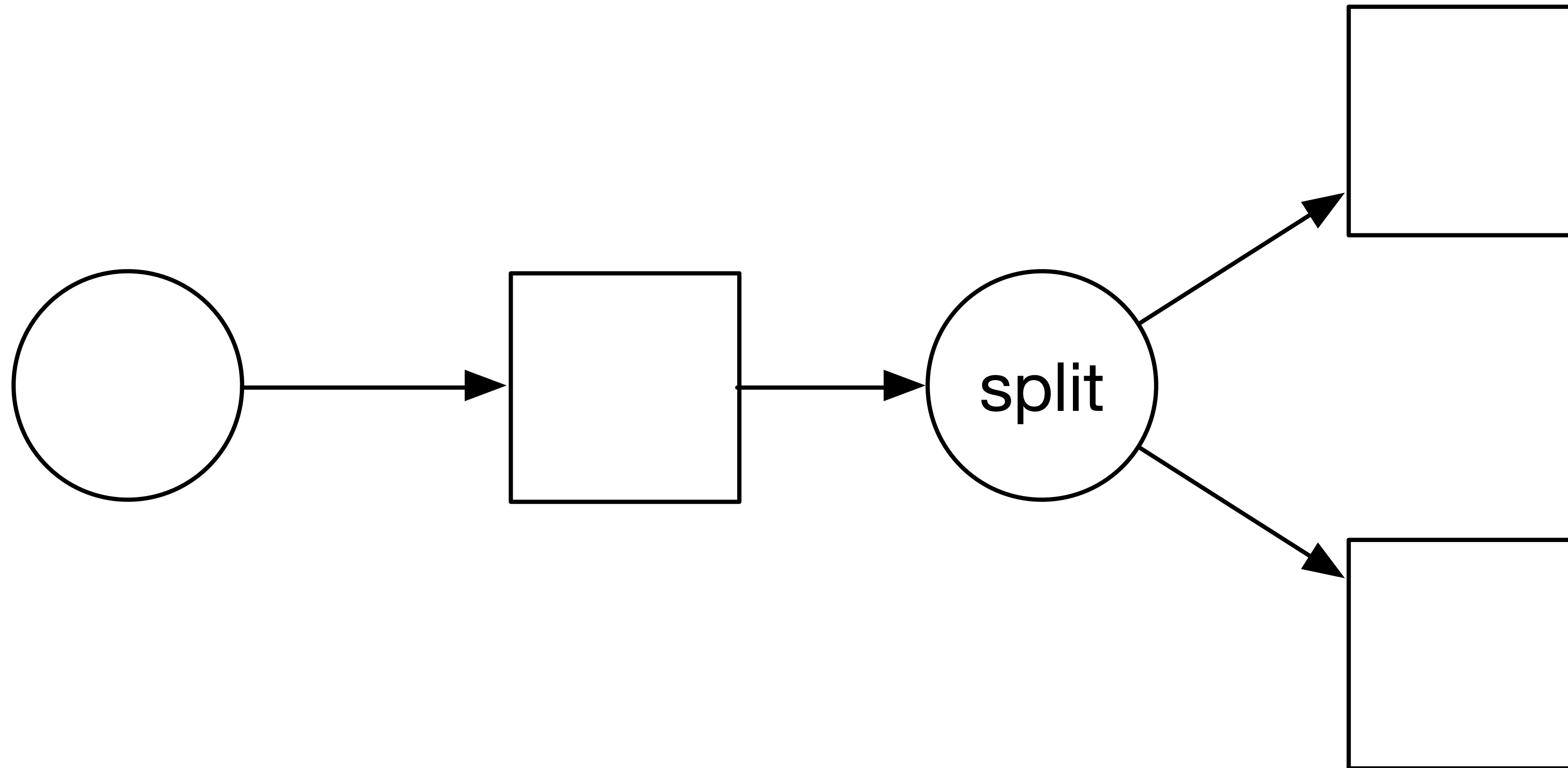
```
future<cpp_int> z = x.then([](future<cpp_int> x){ return cpp_int(x.get() / 15); });
```

Thread 1: signal SIGABRT

Assertion failed: (px != 0), function operator->, file shared_ptr.hpp, line 648.

- Desired behavior
 - A future should behave as a *regular* type - a token for the actual value
 - `shared_futures` let me “copy” them around and do multiple `get ( )` operations
 - But not multiple continuations (at least with Boost as of 1.62, the TS is ambiguous here!)

- We can write a pseudo-copy, `split()`.



Futures: Continuations

```
future<cpp_int> x = async([]{ return fibonacci<cpp_int>(100); });  
  
future<cpp_int> y = split(x).then([](future<cpp_int> x){ return cpp_int(x.get() * 2); });  
future<cpp_int> z = x.then([](future<cpp_int> x){ return cpp_int(x.get() / 15); });  
  
future<void> done = when_all(std::move(y), std::move(z)).then([](auto f){  
    auto t = f.get();  
    cout << get<0>(t).get() << endl;  
    cout << get<1>(t).get() << endl;  
});  
  
done.wait();
```

```
708449696358523830150  
23614989878617461005
```

- Promise is the sending side of a future
- Promises are packaged with a function to form a *packaged task*
 - Packaged tasks handle the exception marshalling through a promise

Promise

```
promise<int> x;  
future<int> y = x.get_future();  
  
x.set_value(42);  
cout << y.get() << endl;
```

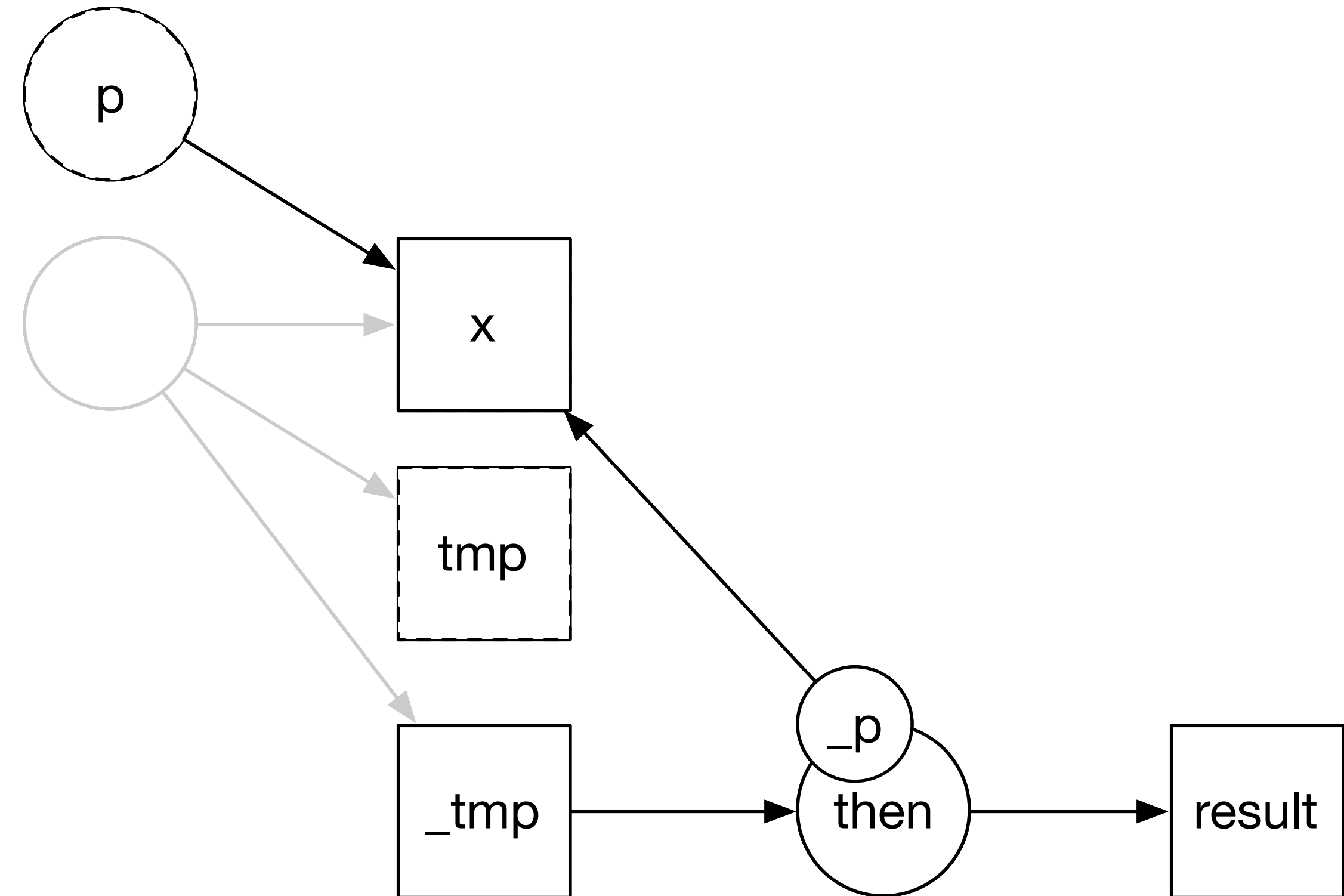
42

Futures: Split

```
template <typename T>
auto split(future<T>& x) {
    auto tmp = std::move(x);

    promise<T> p;
    x = p.get_future(); // replace x with new future

    return tmp.then([_p = move(p)](auto _tmp) mutable {
        auto value = _tmp.get();
        _p.set_value(value); // assign to new "x" future
        return value; // return value through future result
    });
}
```



Futures: Split

```
template <typename T>
auto split(future<T>& x) {

    auto tmp = std::move(x);

    promise<T> p;
    x = p.get_future(); // replace x with new future

    return tmp.then([_p = std::move(p)](auto _tmp) mutable {
        if (_tmp.has_exception()) {
            auto error = _tmp.get_exception_ptr();
            _p.set_exception(error);
            rethrow_exception(error);
        }

        auto value = _tmp.get();
        _p.set_value(value); // assign to new "x" future
        return value; // return value through future result
    });
}
```

Futures: Continuations

```
future<cpp_int> x = async([]{ return fibonacci<cpp_int>(100); });

future<cpp_int> y = split(x).then([](future<cpp_int> x){ return cpp_int(x.get() * 2); });
future<cpp_int> z = x.then([](future<cpp_int> x){ return cpp_int(x.get() / 15); });

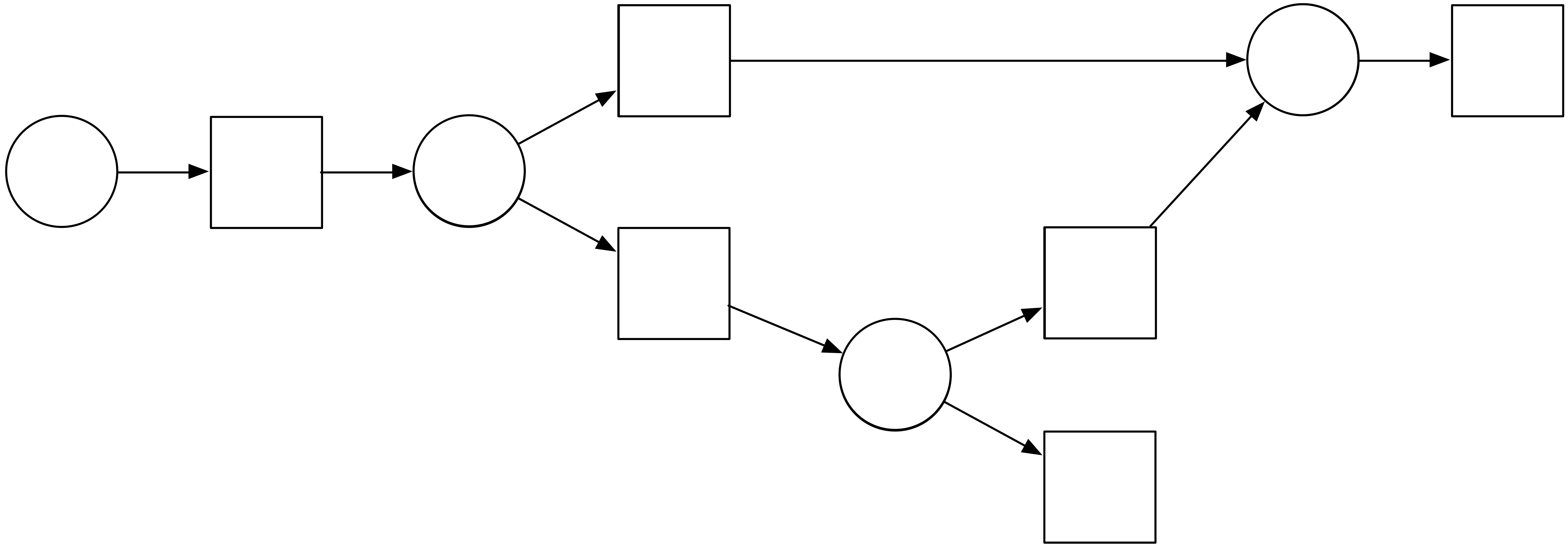
future<void> done = when_all(std::move(y), std::move(z)).then([](auto f){
    auto t = f.get();
    cout << get<0>(t).get() << endl;
    cout << get<1>(t).get() << endl;
});

done.wait();
```

```
708449696358523830150
23614989878617461005
```


- When the (last) future destructs
 - The associated task that has not started, should not execute (NOP)
 - The resource held by that task should be released
 - Since that task may hold futures for other tasks, the system unravels
- I do not know of a good way to compose such cancelation with current futures

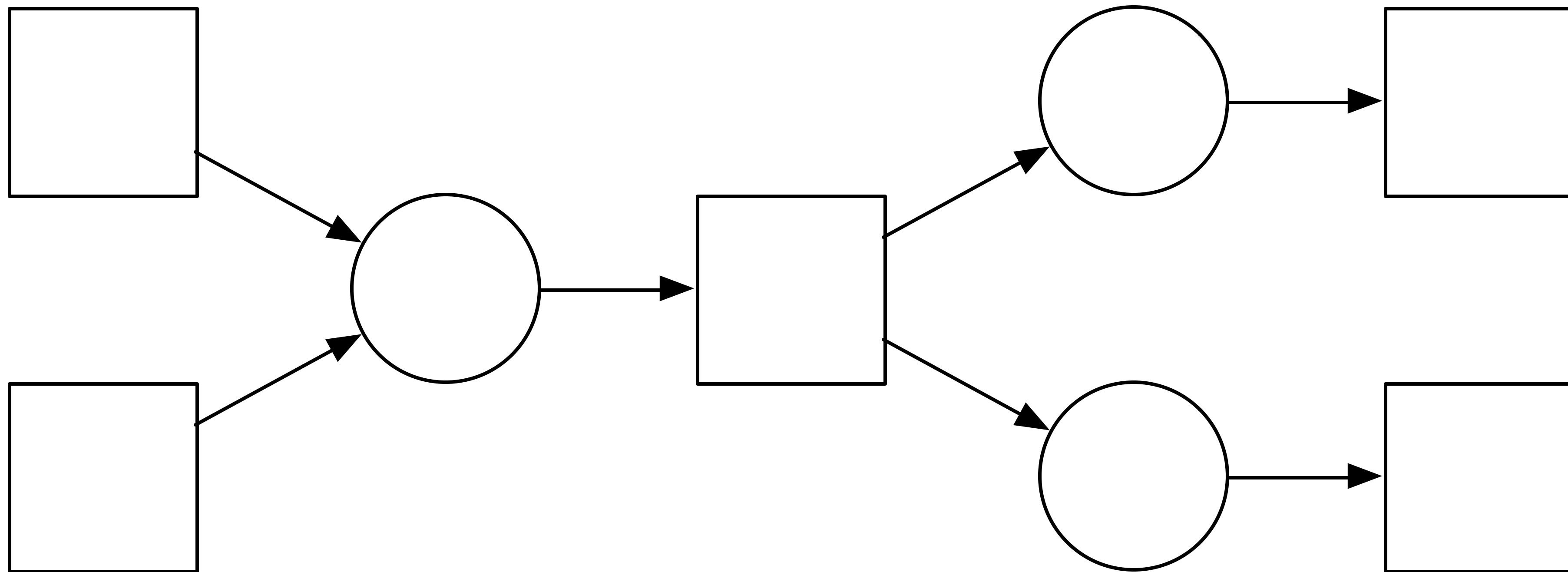
Cancellation



- Currently supports
 - Multiple continuations and copy
 - Optimized for rvalues
 - Join (When All, When Any)
 - Cancellation on Destruction (and explicit reset)
 - And detach
- <https://github.com/stlab/libraries/tree/develop>
- Thanks to Felix Petriconi

Channels

What if we persist the graph?



What if we persist the graph?

- Allow multiple invocations of the tasks by setting the source values
- Each change triggers a notification to the sink values
- This is a reactive programming model and futures are known as *behaviors* or *channels*

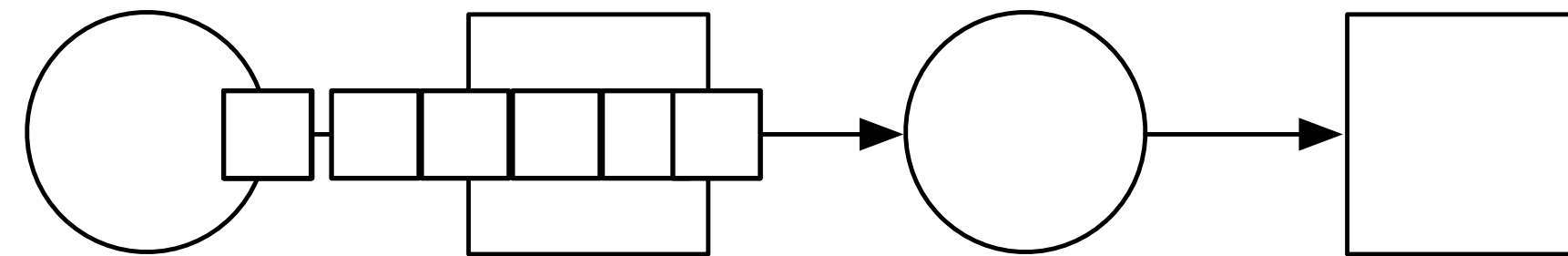
Accumulators and Generator

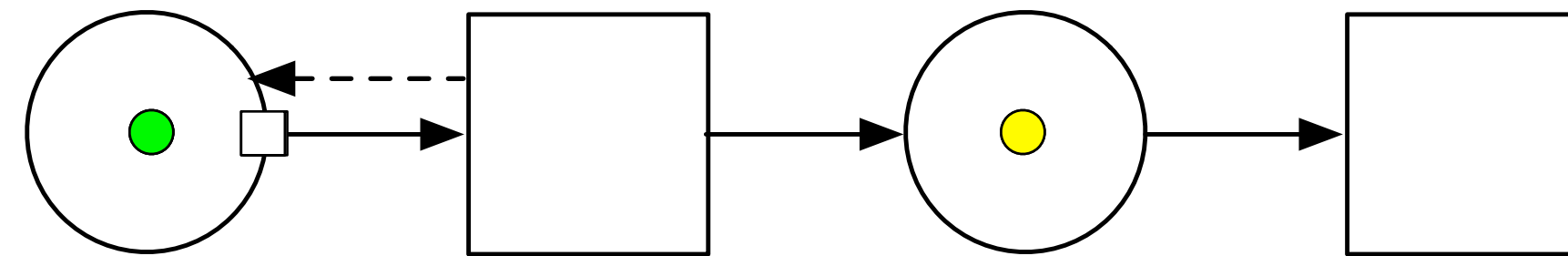
- Each operation does not have to be a 1:1 mapping of input to output
- Coroutines are one way to write n:m functions


```
channel<int> send;

auto hold = send
| [](const receiver<int>& r) {
    int sum = 0;
    while(auto v = co_await r) {
        sum += v.get();
    }
    return sum;
}
| [](int x){ cout << x << '\n'; };

send(1);
send(2);
send(3);
send.close();
```



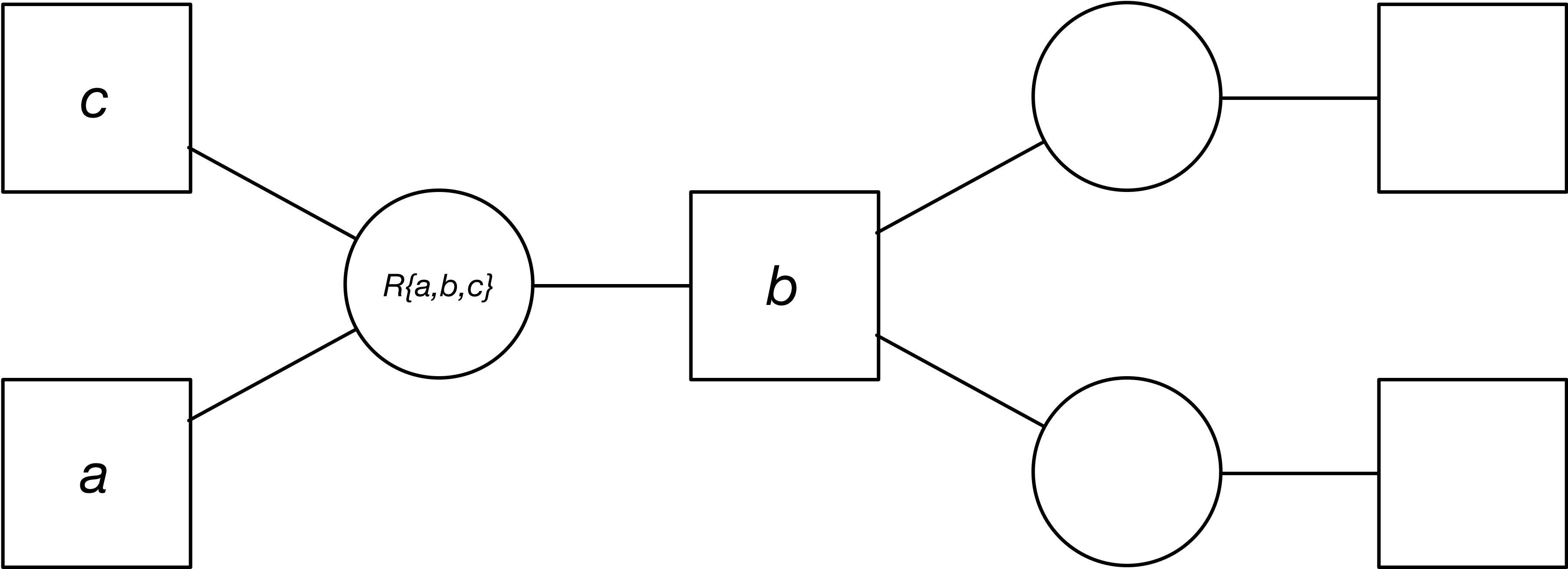


Channels

```
task<void> _render(channel<parameters> in) {  
    while (true) {  
        optional<parameters> params;  
        while (params = co_await in) {  
            co_yield render_frame(*params, draft_quality);  
        }  
        co_yield render_frame(*params, final_quality);  
    }  
}
```

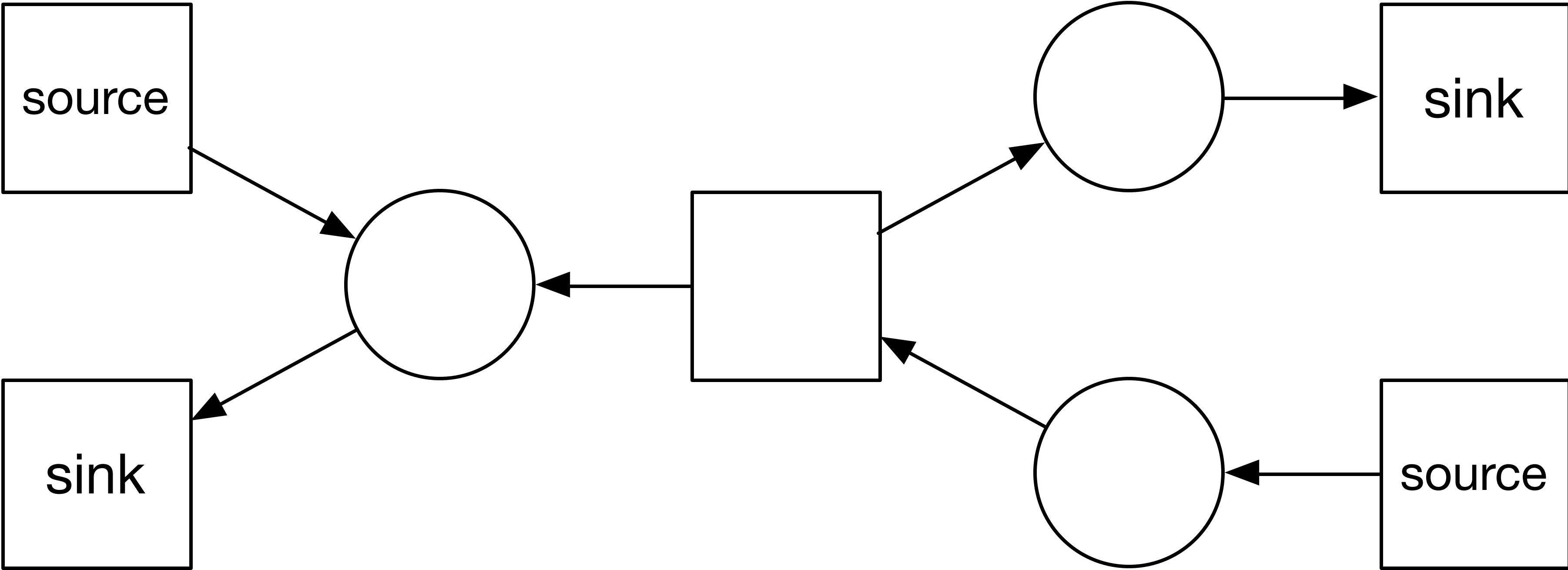

Property Models

How do the graphs change during execution?



A function is a directed relationship

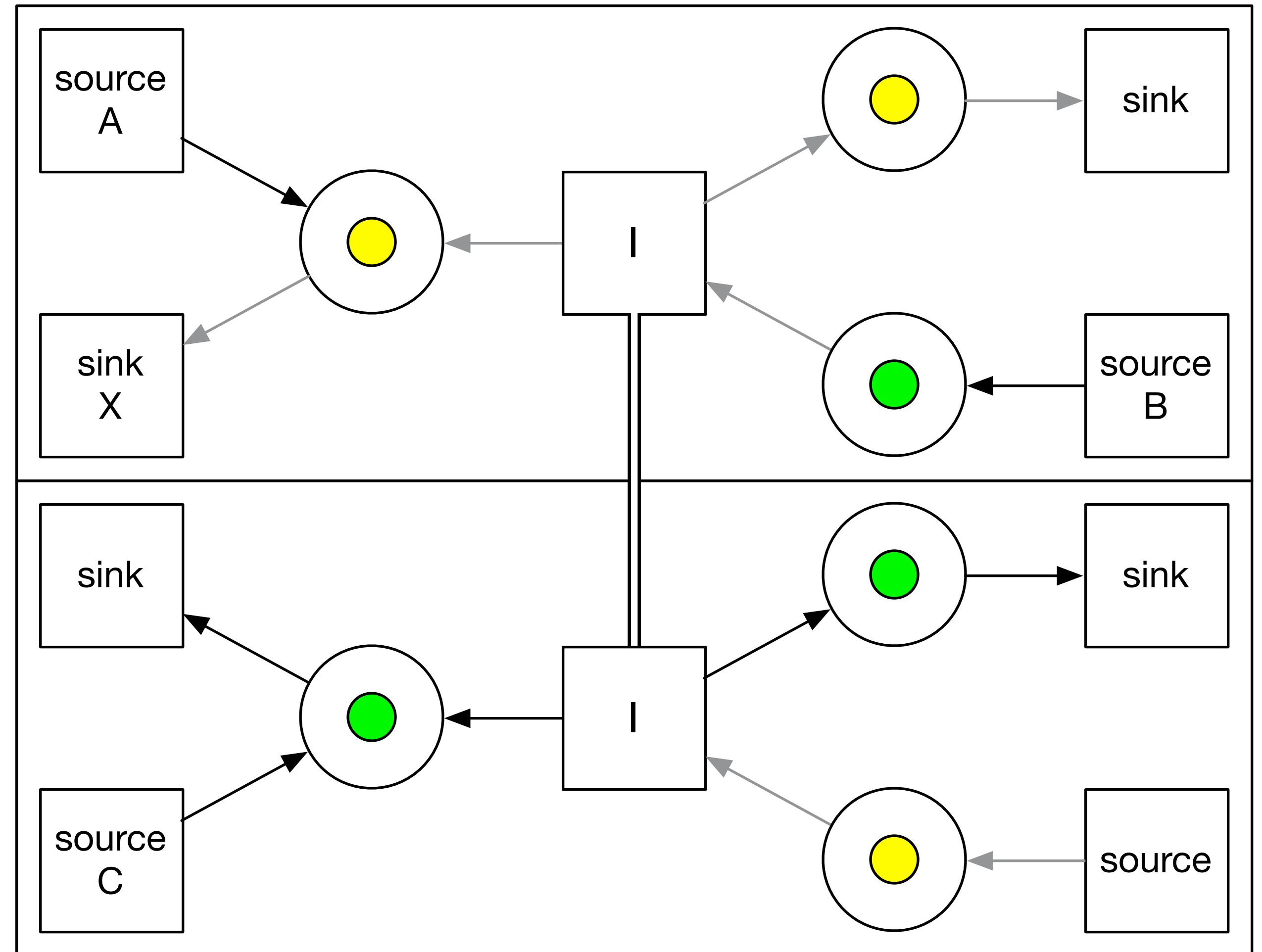
- We can remove the arrows by providing a package of functions to represent the relationship
 - $a = b * c$
 $b = a / c$
 $c = a / b$
 - This forms a type of constraint system called a *property model*
 - Flow is determined by value, or *cell*, priority
 - Relationships can be conditional, so long as predicate can be determined regardless of flow
- Cells can only have one in-edge for a given flow or the system is over constrained

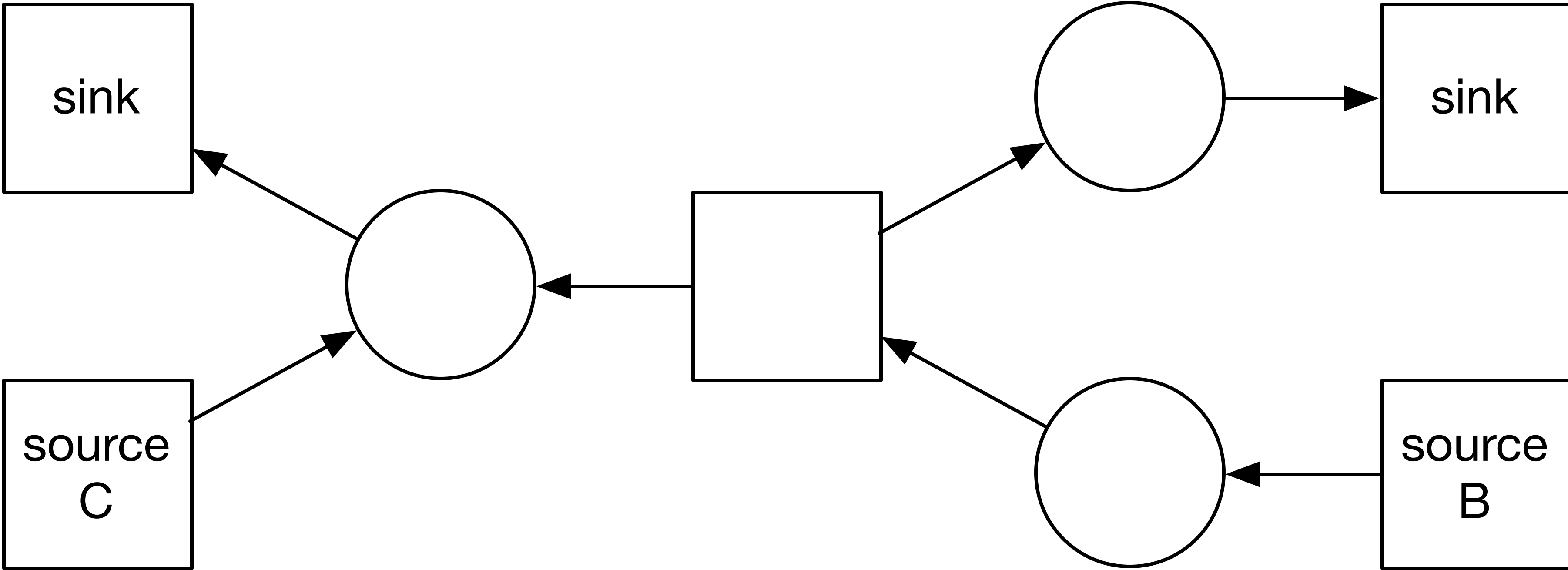


- Reflowing a property model doesn't require all relationships to be resolved
 - The task representing them can still be executing concurrently
- This creates a single dependency graph that is appended to for each new flow and is pruned and *unravels* as tasks are complete

Property Model

- Value set in source A then B
- Operation connected to B is high latency
- Value is set in source C causing reflow
 - Sink X is no longer needed, pending operation is canceled
 - Source A is discarded
- Intermediate value I is shared between flows, once determined
- Final values determined by source B (via I) and source C





- Very useful for UI behavior
- Significant information is in the graph
 - Source / Derived values form a partition set
 - Easily model checked
 - Equal result regardless of source order
 - Form an operational transform, useful for collaborative editing
 - A value is *implied* by the current state it only has in in-edge
 - Source values determine *intent*
 - Values disconnected from result (sink) are *don't care*

- Perhaps representing such systems *as if* it where imperative code is not the correct approach
- Instead a graph description can be compiled and statically validated
- Slides and code from talk (and property model papers):
- <http://sean-parent.stlab.cc/papers-and-presentations>
- Experimental future and channel library:
- <https://github.com/stlab/libraries/tree/develop>
- Thanks to Felix Petriconi
- Communicating Sequential Processes (C. A. R. Hoare)
- <http://usingcsp.com/cspbook.pdf>



Adobe